Project Report
on

NETSCANNER : WEB BASED NETWORK SCANNER

at

LetsDefend Infosec

Submitted in partial fulfillment of the requirement
for the award of the degree of
Bachelor of Technology
in
COMPUTER ENGINEERING
by

Purvam Patel (22012011036)

Under the Guidance of

Internal Guide:
Prof. Prachi Shah

External Guide:
Mr. Kishan Patel

Semester VIII

Submitted to
Department of Computer Engineering
Faculty of Engineering & Technology
Ganpat University, Ganpat Vidyanagar-384012, Gujarat
April- 2026

U.V. PATEL COLLEGE OF ENGINEERING

Ganpat Vidyanagar-384012, Mehsana-Gandhinagar Highway,
District-Mehsana, Gujarat, INDIA

CERTIFICATE

This is to certify that **Mr. Purvam Patel** student of **B.Tech. Semester VIII (Computer Engineering)** has completed his full semester project work titled "**Netscanner**" satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering of Ganpat University, Ganpat Vidyanagar in the year 2025-2026.

Prof. Prachi Shah
Internal Guide

Dr. Paresh Solanki
Head of the Department
Computer Engineering

Stamp

Date of Examination: 25/04/2026

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this Software Requirements Specification (SRS) document.

First and foremost, I would like to thank my company guide, **Mr. Kishan Patel**, for his invaluable guidance, continuous support, and practical insights throughout the course of this project. His mentorship played a crucial role in shaping the direction and execution of this work.

I would also like to express my heartfelt thanks to our Head of Department, **Dr. Paresh Solanki**, for his encouragement and support, which greatly motivated me during the development of this project.

I am especially grateful to my internal guide, **Prof. Prachi Shah**, for her guidance, constructive feedback, and constant encouragement. Her valuable suggestions significantly contributed to improving the quality of this document.

I am deeply thankful to the professors and mentors at **U. V. Patel College of Engineering (UVPCE), Ganpat University**, for providing the academic foundation and supportive environment necessary for this work. I would also like to thank my peers and friends for their thoughtful discussions and collaborative spirit, which helped me gain new perspectives and enhance the quality of this project.

I extend my appreciation to the authors of various references and resources that contributed to the development of this document. Their work provided valuable insights and direction for my understanding of the subject.

Best Regards,
Purvam Patel

ABSTRACT

With the increasing complexity of modern computer networks, effective monitoring and analysis have become essential for maintaining performance, reliability, and security. Traditional network scanning tools are powerful but often rely on command-line interfaces and generate complex outputs that are difficult for beginners and non-technical users to interpret.

This project presents a **Web-Based Network Scanning and Analysis Dashboard** designed to simplify network scanning through a user-friendly graphical interface. The system allows users to configure scans, execute them easily, and view results in structured formats such as tables and visual summaries. It provides key information including open ports, detected services, and protocol details, helping users better understand network conditions and identify potential security concerns.

The platform is especially useful for educational environments and cybersecurity training, as it promotes safe and ethical scanning within authorized networks. Overall, the system improves accessibility, usability, and understanding of network security practices through clear visualization and simplified interaction.

INDEX

ABSTRACT i

1. INTRODUCTION 1

1.1. Project Overview 1

1.2. Project Background 1

1.3. Purpose 2

1.4. Problem Statement 3

1.5. Tools And Technologies 3

2. PROJECT SCOPE 5

2.1. Core Features 5

2.2. Objective 5

2.3. Feature and Functionality 6

2.4. User Interface and User Experience (UI/UX) 6

2.5. Security and Compliance 6

2.6. Scalability and Performance 7

2.7. Testing and Quality Assurance 7

3. LITERATURE SURVEY 8

3.1. Introduction 8

3.2. Existing Network Scanning Tools 8

3.3. Comparative Analysis of Existing Tools 9

3.4. Research Gap Identified 10

3.5. Need for Web-Based Network Scanner 11

4. FEASIBILITY ANALYSIS 13

4.1. Technical Feasibility 13

4.2. Time Schedule Feasibility 13

4.3. Operational Feasibility 14

4.4. Implementation Feasibility 14

4.5. Economic Feasibility 15

5. SOFTWARE & HARDWARE REQUIREMENTS 16

5.1. Software Requirements 16

5.2. Hardware Requirements 17

6. PROCESS MODEL 18

6.1. Introduction 18

6.2. Justification for Choosing Agile Incremental Model 18

6.3. Process Flow (Agile-Incremental Development) 19

6.4. Process Model Diagram 20

6.5. Advantages of This Model for Web-Based Network Scanner 20

7. PROJECT PLAN 22

7.1. Project Milestones 22

7.2. Work Breakdown Structure (WBS) 22

7.3. Resource Allocation 23

7.4. Risk Management 24

8. SYSTEM DESIGN 26

8.1. Use Case Diagrams 26

8.2. Class Diagram 27

8.3. Sequence Diagram 28

8.4. Activity Diagram 29

8.5. DFD (Data Flow Diagram) 30

8.5.1. DFD 0 30

8.5.2. DFD 1 30

8.5.3. DFD 2 31

8.6. User Interface 32

9. IMPLEMENTATION DETAILS 39

9.1. Algorithm of Implementation 39

9.2. Flowchart of Implementation 41

9.3. Actual Program Code 42

10. TESTING 53

10.1. Testing Approach 53

10.2. Testing Model Used 53

10.3. Types of Testing Performed 54

10.4. Test Cases and Results 54

10.5. Test Results Summary 55

10.6. Conclusion of Testing 56

11. USER MANUAL 57

11.1. About Web-Based Network Scanner 57

11.2. System Requirements 58

11.3. User Types 59

11.4. Registration (New Users) 59

11.5. Login 60

11.6. User Portal Guide 60

11.7. Network Scanning and Analysis Features 61

11.8. Troubleshooting 62

12. CONCLUSION & FUTURE WORK 65

12.1. Conclusion 65

12.2. Future Work 66

13. ANNEXURE 68

13.1. Glossary of Terms and Abbreviations 68

13.2. References 69

13.3. About Tools and Technology 69

13.4. About The Organization 71

13.5. About College 74

LIST OF FIGURES

Figure 8-1 Use Case Diagram 26

Figure 8-2 Class Diagram 27

Figure 8-3 Sequence Diagram 28

Figure 8-4 Activity Diagram 29

Figure 8-5 DFD-0 30

Figure 8-6 DFD-1 30

Figure 8-7 DFD-2 31

Figure 8-8 Login Page 32

Figure 8-9 Sign Up Page 33

Figure 8-10 Dashboard 34

Figure 8-11 Result Page 35

Figure 8-12 Host Discovery Page 36

Figure 8-13 Log Analyzer Page 37

Figure 8-14 Port Scan Page 38

Figure 9-1 Flowchart 41

LIST OF TABLES

Table 3-1 Comparative Analysis of Existing Tools 9

Table 7-1 Project Milestones 22

Table 7-2 Risk Management 24

Table 10-1 Test Cases 54

Table 13-1 Glossary of Terms and Abbreviation 68

1. INTRODUCTION

1.1. Project Overview

The **Web-Based Network Scanning and Analysis Dashboard** is a web application designed to provide a simplified, efficient, and user-friendly platform for performing network scanning and monitoring tasks. The primary objective of the system is to automate network scanning operations and present the results through an interactive graphical interface, thereby eliminating the need for direct command-line interaction.

The system enables users to configure various scanning parameters, including target host, scan type, and service detection options, through a web-based interface. Based on the user's input, the backend system securely initiates network scans using standard network scanning techniques. The scan results are then processed and converted into structured and meaningful data.

The analyzed data includes key network information such as open ports, running services, and other network-related characteristics. These results are presented on a dynamic dashboard in the form of tables and graphical visualizations, allowing users to easily interpret and analyze network behavior.

The application is designed to operate within a controlled and ethical environment, ensuring that all scanning activities comply with responsible usage practices. It emphasizes security, reliability, and ease of use.

Overall, the project integrates fundamental networking concepts, backend automation, and web-based visualization techniques. It serves as a valuable tool for both academic learning and practical understanding of network scanning and analysis.

1.2. Project Background

Modern computer networks have become increasingly complex due to the rapid growth of connected devices, applications, and services. Organizations rely heavily on network infrastructure to support their daily operations, making network visibility and continuous monitoring essential for ensuring performance,

reliability, and security. Network administrators and security professionals commonly use network scanning techniques to identify active hosts, open ports, running services, and potential vulnerabilities or misconfigurations within a network.

Traditional network scanning tools are predominantly command-line based and require a significant level of technical expertise to operate effectively. Although these tools provide detailed and accurate information, their raw outputs are often difficult to interpret, particularly for students, beginners, and non-technical users. This limitation creates a need for a more accessible system that simplifies network scanning without compromising on functionality or accuracy.

A web-based network scanning and analysis platform addresses these challenges by offering an intuitive and user-friendly interface for configuring scan parameters and visualizing results. By presenting scan outputs in a structured and graphical format, the system enhances usability and improves understanding of network behavior. Additionally, it reduces dependency on command-line knowledge and promotes ethical and controlled usage of network scanning techniques.

This project is motivated by the need to bridge the gap between complex network scanning tools and user-friendly analysis systems. It aims to provide an effective solution that facilitates better learning, understanding, and management of network environments for both academic and practical purposes.

1.3. Purpose

The purpose of this project is to design and develop a **Web-Based Network Scanner** that enables users to discover devices, identify open ports, and analyze services within a network through a centralized and user-friendly web interface.

The system is intended to assist students, network administrators, and security professionals in gaining better visibility into network structures and behavior. It helps users understand network exposure by providing detailed insights into active hosts, open ports, and running services.

Additionally, the system aims to support the identification of potential security risks in a controlled and ethical manner. By simplifying network scanning

operations and presenting results in an accessible format, the application enhances learning, improves usability, and promotes responsible use of network scanning techniques.

1.4. Problem Statement

With the increasing complexity of modern computer networks, identifying active hosts, open ports, and exposed services has become a challenging task. Effective network monitoring and analysis are essential for maintaining security, performance, and reliability.

Traditional network scanning tools are primarily command-line based and require significant technical expertise to operate. These tools often lack centralized access, structured reporting, and user-friendly interfaces, making them difficult to use for beginners and non-technical users. Additionally, the output generated by such tools can be complex and hard to interpret.

The absence of an intuitive and centralized network scanning platform can lead to issues such as misconfigurations, undetected vulnerabilities, and inefficient network monitoring practices. Therefore, there is a need for a web-based solution that simplifies network scanning processes while presenting results in a clear, structured, and easily understandable format.

1.5. Tools And Technologies

The proposed system is developed using modern and widely adopted open-source tools and technologies to ensure efficiency, scalability, and maintainability. The technology stack used in the project is as follows:

- **Programming Language: Python**
  Used for implementing backend logic, network scanning operations, and data processing.
- **Web Framework: Flask / Django**
  Provides the foundation for building the web-based application and handling server-side functionality.

- **Networking Libraries: Scapy, Socket Programming, PyShark**
  Utilized for performing network scanning, packet analysis, and service detection.
- **Database: SQLite / PostgreSQL**
  Used for storing scan results, user data, and system logs in a structured format.
- **Frontend Technologies: HTML, CSS, JavaScript**
  Responsible for designing the user interface and creating interactive dashboard components.
- **Development Tools: VS Code / Cursor IDE**
  Used for coding, debugging, and managing the development process.
- **Version Control: Git and GitHub**
  Enables version tracking, collaboration, and source code management.

2. PROJECT SCOPE

2.1. Core Features

The system provides the following core features to facilitate efficient network scanning and analysis:

- **Network Host Discovery:**
  The system identifies active devices within a specified network by detecting reachable hosts.
- **Port Scanning:**
  Enables scanning of target systems to identify open, closed, and filtered ports, helping in understanding network exposure.
- **Service and Protocol Identification:**
  Detects services running on open ports along with the associated protocols, providing deeper insight into the network configuration.
- **Web-Based Visualization:**
  Displays scan results through an interactive web dashboard using tables and graphical representations for better understanding and analysis.
- **User Authentication and Role-Based Access:**
  Ensures secure access to the system by implementing authentication mechanisms and restricting functionalities based on user roles (e.g., admin, user).

2.2. Objective

The primary objective of this project is to design and develop a secure and efficient web-based platform that enables users to perform network scanning operations and analyze network exposure with minimal technical complexity.

The system aims to simplify the process of network analysis by providing an intuitive interface, allowing users to easily configure scanning parameters and interpret results. It also focuses on enhancing usability while maintaining accuracy and reliability in scanning operations.

Additionally, the project seeks to promote ethical and controlled use of network scanning techniques, ensuring that all activities are conducted within a secure and authorized environment.

2.3. Feature and Functionality

- Scan networks for live hosts
- Identify open, closed, and filtered ports
- Display service details for open ports
- Store scan results for future reference
- Provide administrative control and monitoring

2.4. User Interface and User Experience (UI/UX)

The system provides a visually engaging and intuitive web-based user interface designed using a **cyberpunk-themed template**. The interface incorporates a modern, futuristic design with neon color schemes, high-contrast elements, and dynamic visual components to enhance user interaction and overall experience.

The UI allows users to easily initiate network scans, view detailed results, and navigate through reports with minimal effort. Despite its advanced visual design, the interface maintains simplicity and usability, ensuring that users with varying levels of technical expertise can operate the system effectively.

Information is presented in a clear and structured format using tables, panels, and graphical visualizations integrated within the cyberpunk layout. The design also ensures responsiveness, enabling seamless access across different devices such as desktops, tablets, and mobile platforms.

Overall, the cyberpunk-themed interface not only improves aesthetic appeal but also enhances user engagement while maintaining clarity, accessibility, and functionality.

2.5. Security and Compliance

- Role-based access control
- Secure authentication mechanisms
- Input validation to prevent misuse

- Ethical usage guidelines for authorized scanning only

2.6. Scalability and Performance

The system architecture supports scalability by allowing:

- Expansion of scanning modules
- Database optimization for large scan histories
- Deployment on cloud or distributed environments

2.7. Testing and Quality Assurance

- Unit testing of scanning modules
- Integration testing between backend and frontend
- Performance testing for concurrent scans
- Validation of scan accuracy

3. LITERATURE SURVEY

3.1. Introduction

The literature survey provides an overview of existing research, methodologies, and tools related to network scanning and security assessment. It plays a crucial role in understanding current approaches, identifying limitations, and establishing the foundation for the proposed system.

This section focuses on commonly used techniques such as **host discovery, port scanning, service identification, and network analysis**, which are essential components of network reconnaissance. These techniques are widely used by network administrators and security professionals to monitor network activity, detect vulnerabilities, and ensure the proper functioning of network infrastructures.

The survey also examines how traditional tools implement these methodologies and highlights the challenges associated with their usage, particularly in terms of usability, accessibility, and result interpretation. By analyzing existing solutions, this study identifies gaps that motivate the development of a more user-friendly and efficient web-based network scanning platform.

3.2. Existing Network Scanning Tools

Several network scanning tools are widely used for network reconnaissance, monitoring, and security assessment. These tools offer powerful functionalities such as **port scanning, service detection, operating system identification, and network mapping**, which are essential for analyzing network structures and identifying potential vulnerabilities.

However, most of these tools are primarily designed as **command-line utilities**, requiring users to manually input commands and parameters. This approach demands a certain level of technical expertise and familiarity with networking concepts, making them less accessible to beginners and non-technical users.

Additionally, the output generated by these tools is often **raw and unstructured**, which can be difficult to interpret without prior knowledge. Users may need to

spend additional time analyzing logs or converting outputs into readable formats for better understanding.

Another limitation is the lack of **integrated visualization and centralized management**. Many tools do not provide built-in dashboards or graphical representations of scan results, and they typically operate on local systems without offering web-based accessibility or multi-user support.

Despite their effectiveness and accuracy, these limitations reduce the usability of existing network scanning tools in **academic environments, training scenarios, and collaborative settings**. This highlights the need for a system that combines the power of traditional scanning tools with the simplicity and accessibility of modern web applications.

3.3. Comparative Analysis of Existing Tools

A comparative analysis has been conducted to evaluate the limitations of existing network scanning tools in contrast with the capabilities of the proposed system. This comparison highlights key differences in terms of usability, accessibility, functionality, and user management.

Table 3-1 Comparative Analysis of Existing Tools

Parameter	**Existing Tools**	**Proposed System**
Interface	Primarily command-line based, requiring users to input manual commands.	Web-based graphical interface with interactive components and cyberpunk design.
Ease of Use	Requires technical expertise and familiarity with command-line operations.	User-friendly interface designed for both technical and non-technical users.
Result Visualization	Provides raw, unstructured output that can be difficult to interpret.	Displays structured results using dashboards,

		tables, and graphical visualizations.
Accessibility	Limited to local systems where the tool is installed.	Accessible via web browsers from multiple devices and locations.
User Management	Does not support multi-user environments or access control mechanisms.	Implements user authentication and role-based access control for secure usage.

The analysis clearly indicates that while existing tools are powerful in terms of functionality, they lack usability, accessibility, and centralized management features. These limitations make them less suitable for beginners, students, and collaborative environments.

In contrast, the proposed system overcomes these challenges by integrating a web-based interface, enhanced visualization, and secure user management. This not only improves usability but also expands the applicability of network scanning tools in academic, training, and controlled security assessment scenarios.

3.4. Research Gap Identified

Despite the availability of various network scanning tools, several limitations exist that reduce their effectiveness, especially in academic, learning, and collaborative environments. Most traditional tools are designed with a strong focus on functionality but often overlook usability, accessibility, and centralized management.

One of the major gaps identified is the **lack of web-based centralized access**. Many existing tools operate as standalone applications or command-line utilities, requiring installation and local execution. This limits their accessibility and makes it difficult to manage scanning activities from a unified platform.

Another significant limitation is the absence of **user authentication and role-based access control mechanisms**. Traditional tools generally do not provide

multi-user support, making it challenging to monitor usage, restrict access, or assign different privileges to users such as administrators and general users. This reduces their suitability for institutional or collaborative use.

Additionally, most tools do not offer **scan history storage and reporting capabilities**. The results generated are often temporary or require manual saving, which makes it difficult to track previous scans, perform comparisons, or generate reports for analysis and documentation purposes.

A further gap is the lack of **beginner-friendly interfaces**. Many network scanning tools rely heavily on command-line interactions and produce raw, unstructured output. This creates a steep learning curve for students, beginners, and non-technical users, limiting their adoption in educational settings.

These limitations highlight the need for a modern solution that integrates **usability, accessibility, security, and data management**. The proposed web-based network scanning system addresses these gaps by providing a centralized platform with user authentication, structured data storage, interactive visualization, and an intuitive interface, thereby enhancing its applicability in both academic and practical environments.

3.5. Need for Web-Based Network Scanner

The proposed system focuses on the design and development of a secure, web-based platform for performing network scanning and analysis in a simplified and controlled environment. It aims to bridge the gap between complex command-line network scanning tools and user-friendly web applications by providing an accessible interface for conducting network reconnaissance tasks.

The system enables users to perform essential network scanning operations such as host discovery, port scanning, and service identification through a centralized dashboard. By automating these processes, the platform reduces manual effort and eliminates the need for extensive command-line knowledge. The results of these operations are processed and presented in a structured and visually intuitive format, allowing users to easily interpret network data.

A key aspect of the system is its emphasis on **security and ethical usage**. The platform is designed to operate within authorized environments, ensuring that scanning activities are conducted responsibly. It incorporates user authentication and role-based access control mechanisms to restrict unauthorized usage and maintain system integrity.

The system is particularly suitable for **academic and educational purposes**, where students can gain practical knowledge of networking concepts and security analysis without dealing with the complexities of traditional tools. Additionally, it can be used for **controlled security assessments**, enabling users to identify potential vulnerabilities, misconfigurations, and exposed services within a network.

However, the scope of the system is limited to **authorized and ethical network scanning**. It does not support unauthorized penetration testing, exploitation, or malicious activities. The platform is intended solely for learning, analysis, and monitoring within permitted environments.

Overall, the system provides a balanced combination of functionality, usability, and security, making it a valuable tool for understanding and managing network infrastructures in a safe and controlled manner.

4. FEASIBILITY ANALYSIS

4.1. Technical Feasibility

The proposed system is technically feasible as it is built using well-established, reliable, and widely supported technologies. The use of **Python** as the primary programming language provides strong support for networking operations, automation, and backend processing through its rich ecosystem of libraries such as Scapy, Socket Programming modules, and PyShark.

Web frameworks such as **Flask or Django** enable efficient development of a scalable and secure web-based interface, allowing seamless interaction between users and the backend system. For data storage, **SQLite or PostgreSQL** offers efficient and structured management of scan results, user information, and system logs.

The system is compatible with standard operating systems such as Windows and Linux and does not require any specialized hardware or high-end infrastructure. Additionally, the availability of extensive documentation and community support further strengthens the technical feasibility of the project.

4.2. Time Schedule Feasibility

The project is feasible within the allocated academic timeline. The adoption of the **Agile Incremental Development Model** allows the system to be developed in multiple phases, ensuring continuous progress and flexibility.

Initially, core functionalities such as host discovery, port scanning, and result visualization are prioritized. Advanced features, including service analysis, reporting, and user management, can be implemented in subsequent iterations. This phased approach reduces development risks and allows early testing and validation.

Proper planning, task allocation, and milestone tracking ensure that each phase of the project is completed on time. Regular evaluation of progress further supports adherence to the defined schedule.

4.3. Operational Feasibility

The system is operationally feasible as it is designed with a strong emphasis on usability and accessibility. The web-based interface allows users to access the system through standard web browsers without requiring installation or advanced technical knowledge.

The intuitive design ensures that users can easily perform network scans, configure parameters, and interpret results with minimal training. The integration of **role-based access control** enhances operational security by restricting system functionalities based on user roles, such as administrators and general users.

Administrative features enable monitoring of system usage, management of users, and maintenance of system performance. These capabilities make the system practical and effective for real-world applications, including academic environments and controlled network analysis scenarios.

4.4. Implementation Feasibility

The implementation of the proposed system is feasible due to its **modular architecture**, which divides the system into independent components such as:

- Network scanning module
- Backend processing and logic
- Database management system
- Web-based user interface

Each module can be developed, tested, and integrated independently, reducing complexity and improving maintainability. The use of open-source libraries and frameworks significantly simplifies the development process.

Furthermore, the system can be deployed in various environments, including local servers, institutional networks, or cloud platforms, with minimal configuration. This flexibility ensures ease of implementation and scalability.

4.5. Economic Feasibility

The project is economically feasible as it relies primarily on **open-source technologies and freely available tools**, eliminating the need for expensive licenses or proprietary software.

Development can be carried out using existing hardware resources such as personal computers or institutional systems, minimizing infrastructure costs. Additionally, the use of lightweight frameworks and databases reduces operational expenses.

Since the system does not require specialized equipment or paid services, the overall cost of development, deployment, and maintenance remains low. This makes the project highly suitable for academic purposes and small-scale implementations.

5. SOFTWARE & HARDWARE REQUIREMENTS

This section outlines the software and hardware requirements necessary for the development, deployment, and efficient operation of the proposed Web-Based Network Scanning and Analysis System. These requirements ensure that the system performs reliably and supports all intended functionalities.

5.1. Software Requirements

The system is developed using modern software technologies and requires the following software components for proper functioning:

- **Operating System:**
  The system is compatible with major operating systems such as **Linux (preferred)** and **Windows**. Linux is recommended due to better support for networking tools and permissions required for low-level network operations.
- **Programming Language:**
  Python 3.9 or higher is used for backend development, providing strong support for network programming, automation, and integration with various libraries.
- **Web Framework:**
  Frameworks such as **Flask or Django** are used to build the web application, manage routing, handle user requests, and integrate backend services with the frontend interface.
- **Database Management System:**
  SQLite or PostgreSQL is used for storing scan results, user credentials, logs, and system-related data in a structured and efficient manner.
- **Web Browser:**
  A modern web browser such as **Google Chrome, Mozilla Firefox, or Microsoft Edge** is required to access and interact with the web-based dashboard.
- **Development Tools:**
  Integrated Development Environments (IDEs) such as **VS Code or**

Cursor IDE are used for writing, debugging, and managing the source code.

- **Version Control System:**
  Git and GitHub are used for source code management, version tracking, and collaborative development.

5.2. Hardware Requirements

The system does not require specialized hardware and can operate efficiently on standard computing devices. The minimum and recommended hardware specifications are as follows:

- **Processor:**
  An **Intel i3 processor or higher** is required to handle basic processing tasks. Higher-end processors can improve performance during intensive scanning operations.
- **RAM (Memory):**
  A minimum of **4 GB RAM** is required for smooth operation, while **8 GB RAM or higher is recommended** for better multitasking and faster processing of scan results.
- **Storage:**
  At least **10 GB of free disk space** is required to store application files, scan results, logs, and database records. Additional storage may be required for long-term data retention.
- **Network Interface:**
  A functional **Ethernet or Wi-Fi adapter** is necessary to perform network scanning operations and communicate with target systems.
- **Internet Connectivity:**
  An active internet connection is required for installing dependencies, performing updates, accessing external APIs (if integrated), and testing the application in real-world scenarios.

6. PROCESS MODEL

6.1. Introduction

The development of the **Web-Based Network Scanning and Analysis System** follows the **Agile Incremental Software Development Model**, which combines the adaptability of Agile methodology with the structured approach of incremental development.

In this model, the system is developed in a series of small, manageable increments, where each increment introduces a specific set of functionalities. Instead of building the entire system at once, the development process focuses on gradually enhancing the system by adding new features in each iteration.

This approach allows continuous improvement of the system and ensures that partially functional versions of the application are available at early stages. It also enables developers to identify and resolve issues early in the development cycle. The model is particularly suitable for cybersecurity-related projects, such as network scanning systems, where requirements may evolve as deeper understanding of networking techniques and tools is gained.

6.2. Justification for Choosing Agile Incremental Model

The Agile Incremental Model was selected for this project due to its flexibility, adaptability, and effectiveness in handling evolving requirements. The justification for choosing this model is as follows:

- **Evolving Requirements:**
  The project requirements changed and expanded as the understanding of network scanning techniques, tools, and security considerations improved over time.
- **Modular Development Approach:**
  The system could be logically divided into independent modules such as host discovery, port scanning, service detection, result visualization, and user management, allowing step-by-step development.
- **Early Working Prototypes:**
  Initial versions of the system could be developed and tested early,

enabling validation of core functionalities before proceeding to advanced features.

- **Continuous Feedback Integration:**
  Feedback obtained during each development phase helped refine system features, improve usability, and enhance performance.
- **Risk Reduction:**
  By validating each increment before moving to the next phase, potential issues were identified early, reducing overall project risk.

Overall, this model provided a balance between flexibility and control, ensuring systematic progress while accommodating changes effectively.

6.3. Process Flow (Agile-Incremental Development)

The development process follows a cyclic and iterative approach, where each increment undergoes a complete development cycle. The key stages involved in each iteration are as follows:

- Requirement Planning:
  Identify and define the set of features and functionalities to be implemented in the current increment based on project priorities.
- System Design:
  Develop architectural designs, data flow diagrams, and module-level specifications for the selected features.
- Implementation:
  Code the planned functionalities, such as network scanning logic, backend processing, or frontend interface components.
- Testing:
  Perform thorough testing, including unit testing and integration testing, to ensure that the newly developed features function correctly and integrate smoothly with existing modules.
- Review and Feedback:
  Analyze the performance of the implemented features, identify issues, and collect feedback for further improvement.

- Increment Release:
  Integrate the tested module into the main system, making it part of the working application.

This iterative cycle is repeated until all planned features are implemented and the system is fully developed.

6.4. Process Model Diagram

The Agile Incremental Model can be represented as a sequence of iterative cycles, where each cycle contributes to the gradual development of the complete system. Each iteration includes the following phases:

- Requirement Analysis
- System Design
- Development (Implementation)
- Testing and Validation
- Increment Delivery

Each cycle results in a functional version of the system with additional features. Over multiple iterations, the system evolves into a complete and fully functional application.

This approach ensures continuous delivery of working software and allows improvements to be incorporated at every stage of development.

6.5. Advantages of This Model for Web-Based Network Scanner

The Agile Incremental Model offers several advantages when applied to the development of the Web-Based Network Scanner:

- **Supports Modular Development:**
  Allows independent development of features such as scanning modules, visualization dashboards, and authentication systems.
- **Facilitates Early Testing:**
  Core functionalities like host discovery and port scanning can be tested early, ensuring correctness and reliability.

- **Adaptability to Changes:**
  Enables easy modification of scanning logic and system features based on testing results and feedback.
- **Risk Mitigation:**
  Identifies and resolves issues at early stages, reducing the likelihood of major failures in later phases.
- **Improved System Quality:**
  Continuous testing and refinement enhance overall system performance, usability, and reliability.
- **Efficient Time Management:**
  Ensures that the project progresses steadily and is completed within the given academic timeline.

7. PROJECT PLAN

7.1. Project Milestones

The project was divided into a series of well-defined milestones to ensure structured development and effective tracking of progress. Each milestone represents a significant phase in the project lifecycle and contributes to the overall completion of the system.

Table 7-1 Project Milestones

Milestone	Description
Project Proposal Approval	Finalization of project title, scope, objectives, and approval from the guide.
Requirement Analysis	Identification and documentation of functional and non-functional requirements.
System Design	Preparation of system architecture, UML diagrams, and data flow diagrams.
Implementation Phase	Development of backend modules (scanning logic) and frontend web interface.
Testing Phase	Conducting functional, integration, performance, and security testing.
Documentation	Preparation of SRS, user manual, and final project report.
Final Submission	Demonstration of the system and final evaluation by the review panel.

7.2. Work Breakdown Structure (WBS)

The Work Breakdown Structure (WBS) divides the overall project into smaller, manageable tasks and subtasks. This hierarchical decomposition helps in better planning, execution, and monitoring of project activities.

- **Requirement Analysis**
  - Identification of functional requirements

- Identification of non-functional requirements

- **System Design**
  - Architecture design
  - UML diagram preparation
  - Database schema design
- **Backend Development**
  - Host discovery module implementation
  - Port scanning module development
  - Service detection module integration
  - Logging and data processing mechanisms
- **Frontend Development**
  - Development of web-based dashboard (cyberpunk-themed UI)
  - Implementation of user authentication system
  - Integration of result visualization components (tables, charts)
- **Testing**
  - Unit testing of individual modules
  - Integration testing of combined system components
  - Performance testing for scanning efficiency
- **Documentation**
  - Preparation of Software Requirement Specification (SRS)
  - User manual creation
  - Final project report documentation

This structured breakdown ensures efficient task allocation and helps in tracking progress at each level.

7.3. Resource Allocation

Effective allocation of resources is essential for successful project execution. The resources required for the development of the system are categorized as follows:

Human Resources

- **Project Developer(s):**
  Responsible for system design, coding, testing, and integration of all modules.
- **Project Guide / Supervisor:**
  Provides technical guidance, reviews progress, and ensures adherence to project standards.

Hardware Resources

- Personal computer or laptop for development and testing
- Reliable network connectivity for performing scans
- Local or virtual test network environment for safe and controlled scanning

Software Resources

- Python programming environment for backend development
- Web framework (Flask or Django) for building the web application
- Database systems (SQLite or PostgreSQL) for data storage
- Development tools such as VS Code or Cursor IDE
- Version control system (Git and GitHub) for source code management

7.4. Risk Management

Risk management is an essential aspect of project planning that involves identifying potential risks and implementing strategies to minimize their impact. The following table outlines key risks associated with the project along with their impact and mitigation strategies:

Table 7-2 Risk Management

Risk	**Impact**	**Mitigation Strategy**
Scope Creep	Delay in project completion	Clearly define and control project requirements
Network Scanning Restrictions	Limited or inaccurate scan results	Use controlled test networks and obtain proper permissions

Performance Issues	Slow scanning and delayed results	Optimize scanning logic and use efficient algorithms
Security Vulnerabilities	Risk of data exposure or misuse	Implement secure authentication and input validation
Time Constraints	Incomplete feature implementation	Prioritize core functionalities and follow schedule
API or Library Issues	Failure of certain functionalities	Use stable, well-documented, and widely supported libraries

Proper identification and management of these risks ensured smooth project execution and reduced the likelihood of major setbacks.

8. SYSTEM DESIGN

8.1. Use Case Diagrams

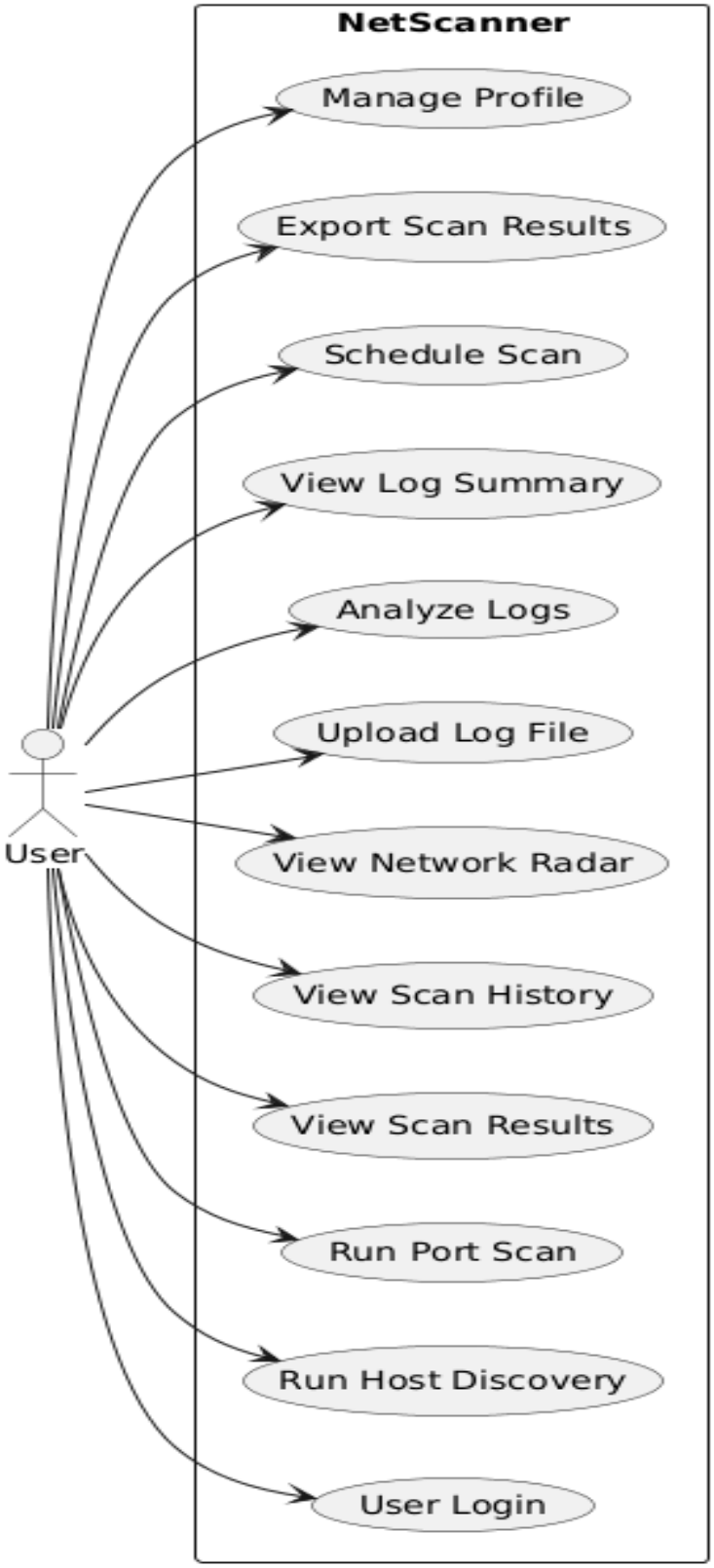

Figure 8-1 Use Case Diagram

8.2. Class Diagram

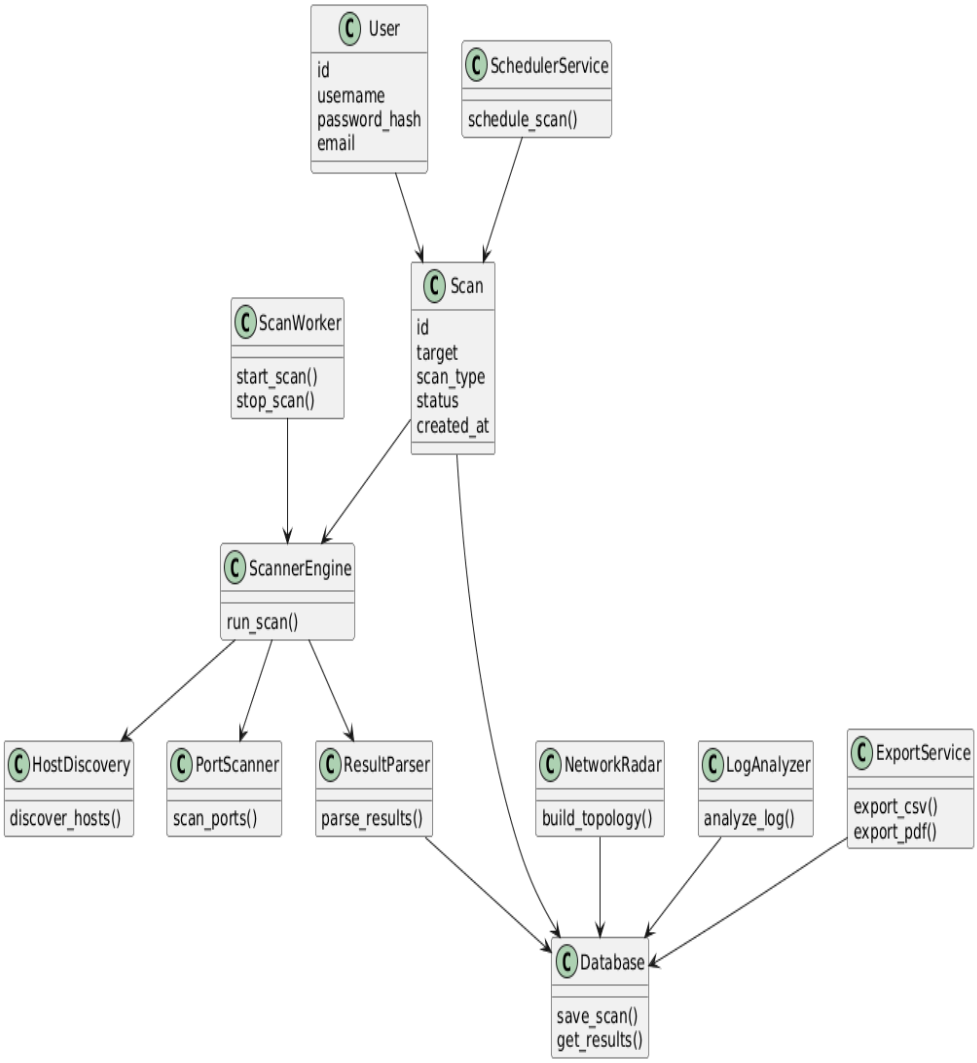

Figure 8-2 Class Diagram

8.3. Sequence Diagram

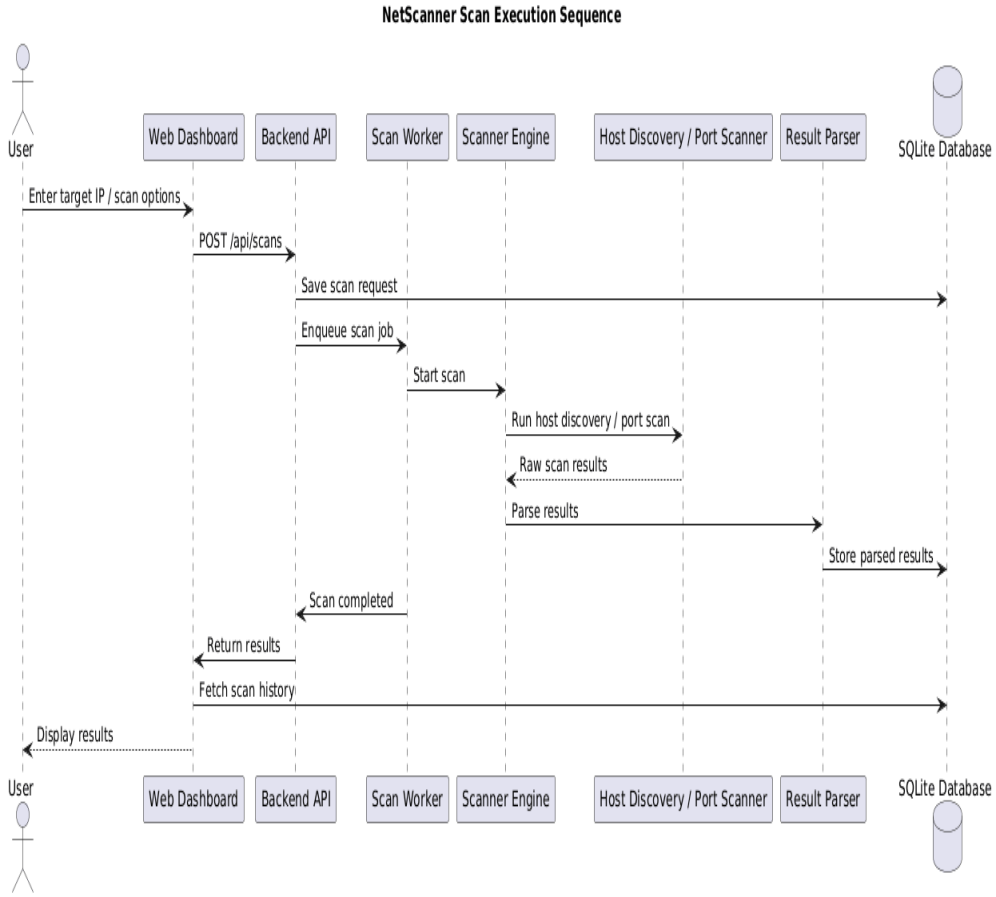

Figure 8-3 Sequence Diagram

8.4. Activity Diagram

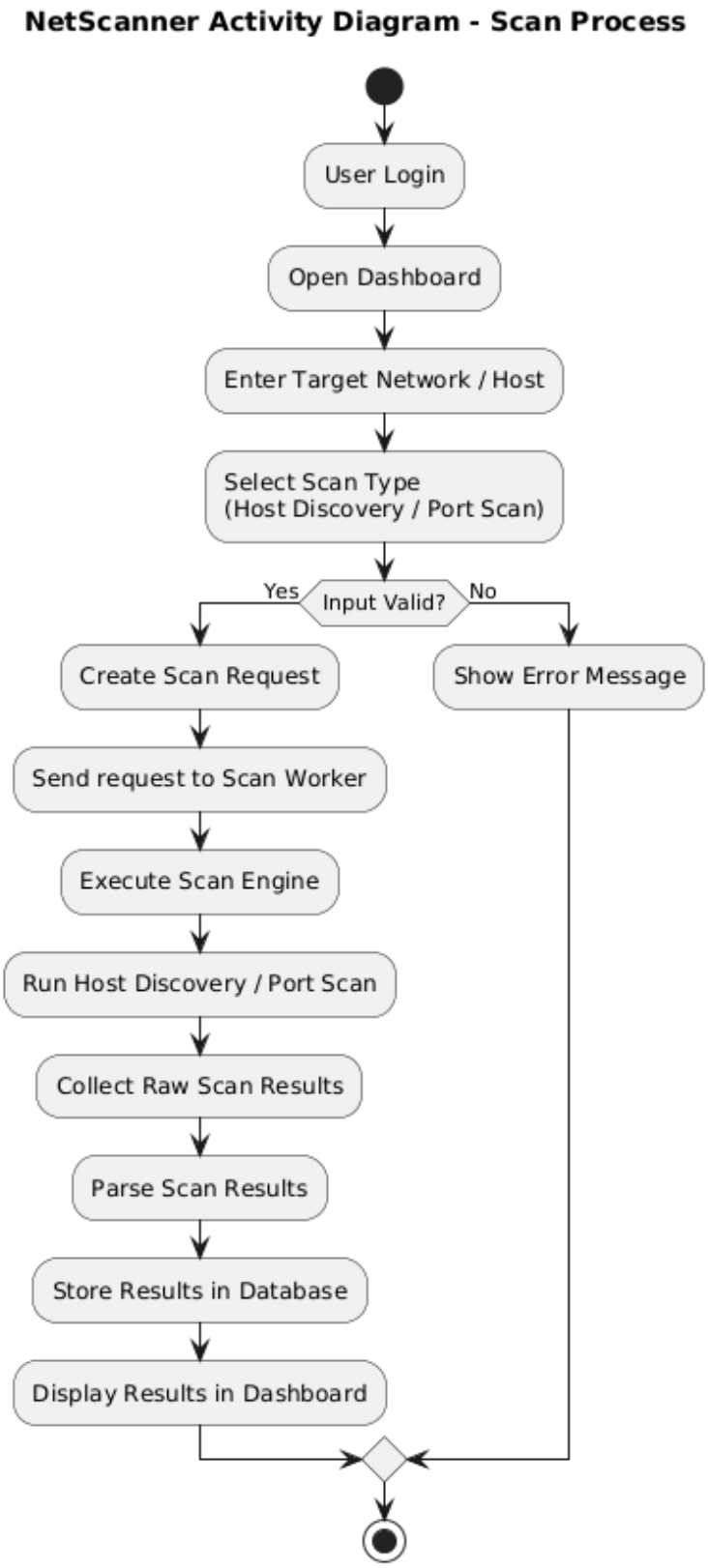

Figure 8-4 Activity Diagram

8.5. DFD (Data Flow Diagram)

8.5.1. DFD 0

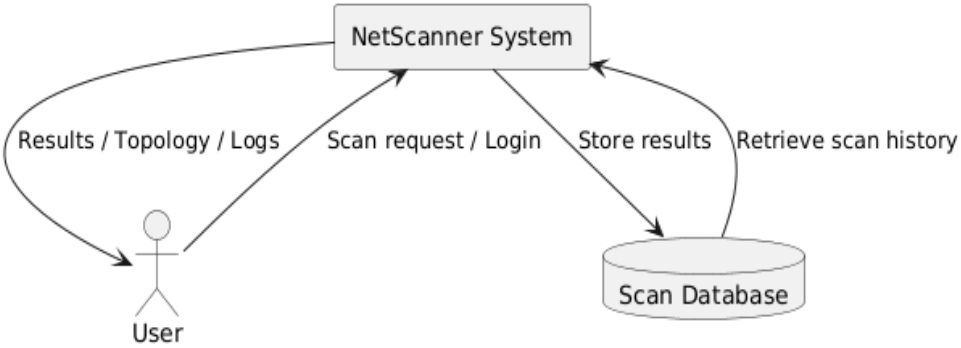

Figure 8-5 DFD-0

8.5.2. DFD 1

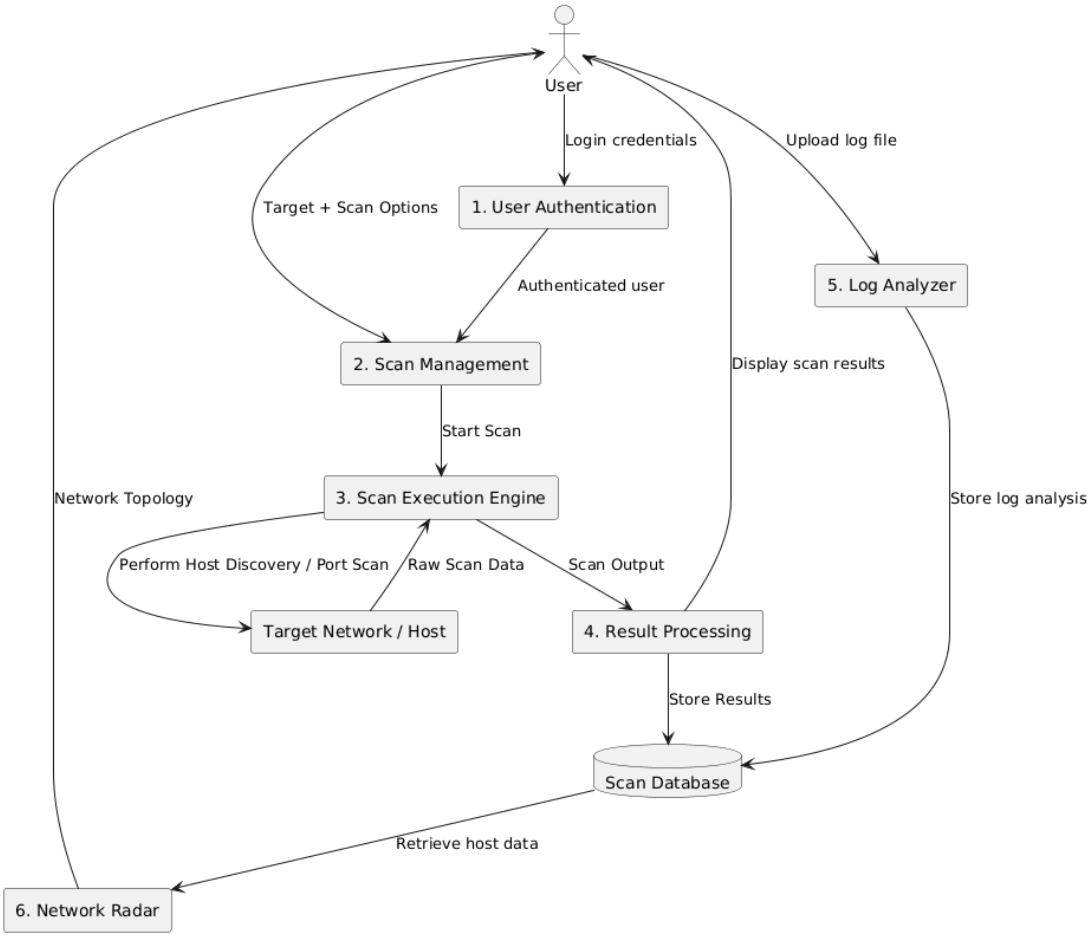

Figure 8-6 DFD-1

8.5.3. DFD 2

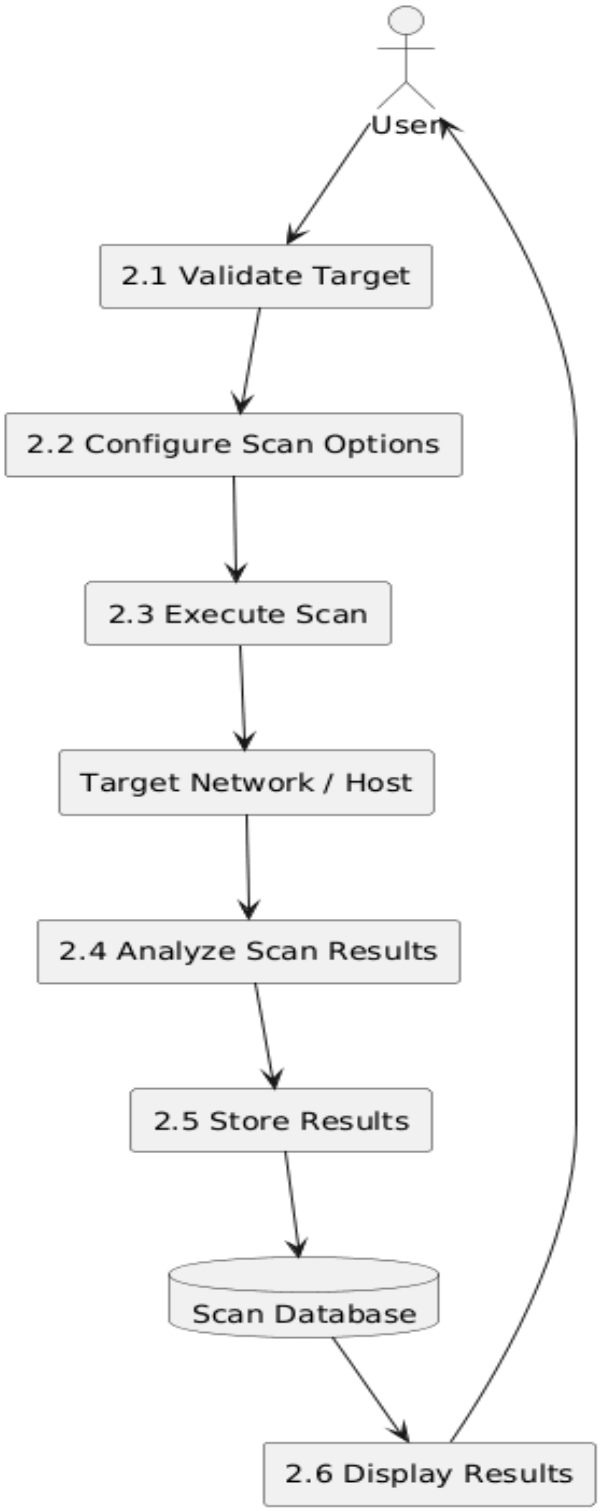

Figure 8-7 DFD-2

8.6. User Interface

- **Login Interface**

  The **Login Interface** of the NetScanner Dashboard serves as the entry point for authorized users to securely access the system. It provides input fields for **Email Address** and **Password**, allowing users to authenticate themselves before entering the dashboard. A **Remember Me** option is included to improve user convenience by maintaining login sessions on trusted devices.

  The interface also includes options such as **Create New Operator Account** for new users and **Reset Password** for account recovery, ensuring smooth user management. The **Access Terminal** button initiates the login process and redirects users to the main dashboard after successful authentication.

  The interface is designed with a clean and modern layout that supports usability, accessibility, and secure access to the network scanning platform.

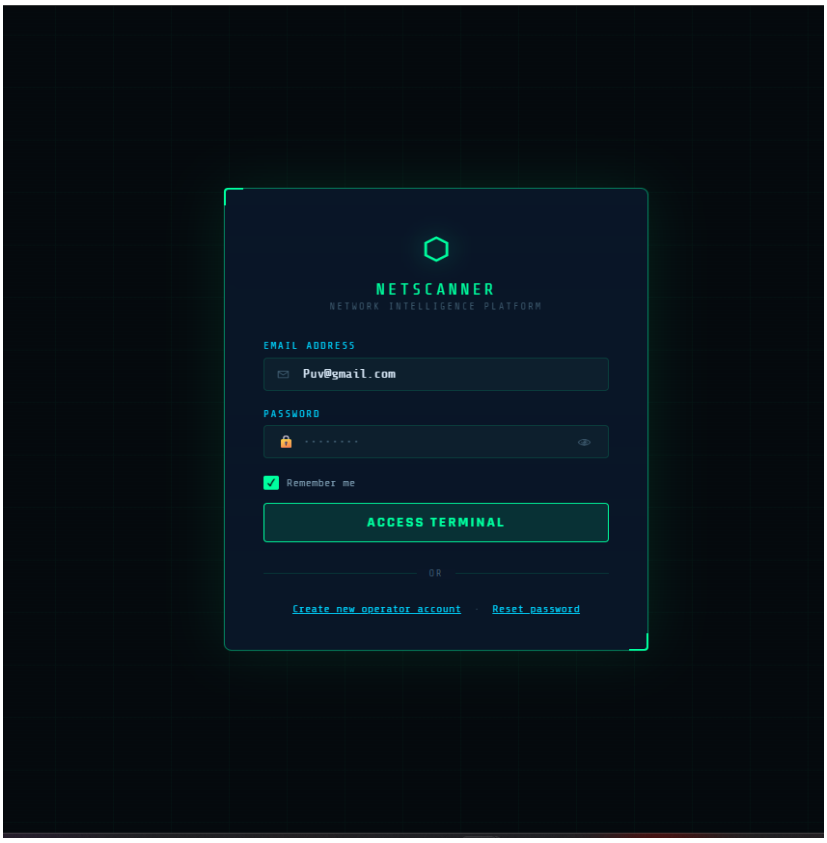

Figure 8-8 Login Page

- **Registration Interface**

  The **Registration Interface** of the NetScanner Dashboard allows new users to create an account and gain authorized access to the system. The interface includes input fields for **Username**, **Email Address**, **Password**, and **Confirm Password**, ensuring that users can securely register their credentials before accessing the platform.

  A minimum password requirement is enforced to improve account security, and the **Create Account** button completes the registration process after validation of the entered information. The interface also provides a **Back to Login** option for existing users who already have an account.

  The registration page is designed with a clear and structured layout that supports secure account creation while maintaining simplicity and ease of use for both technical and non-technical users.

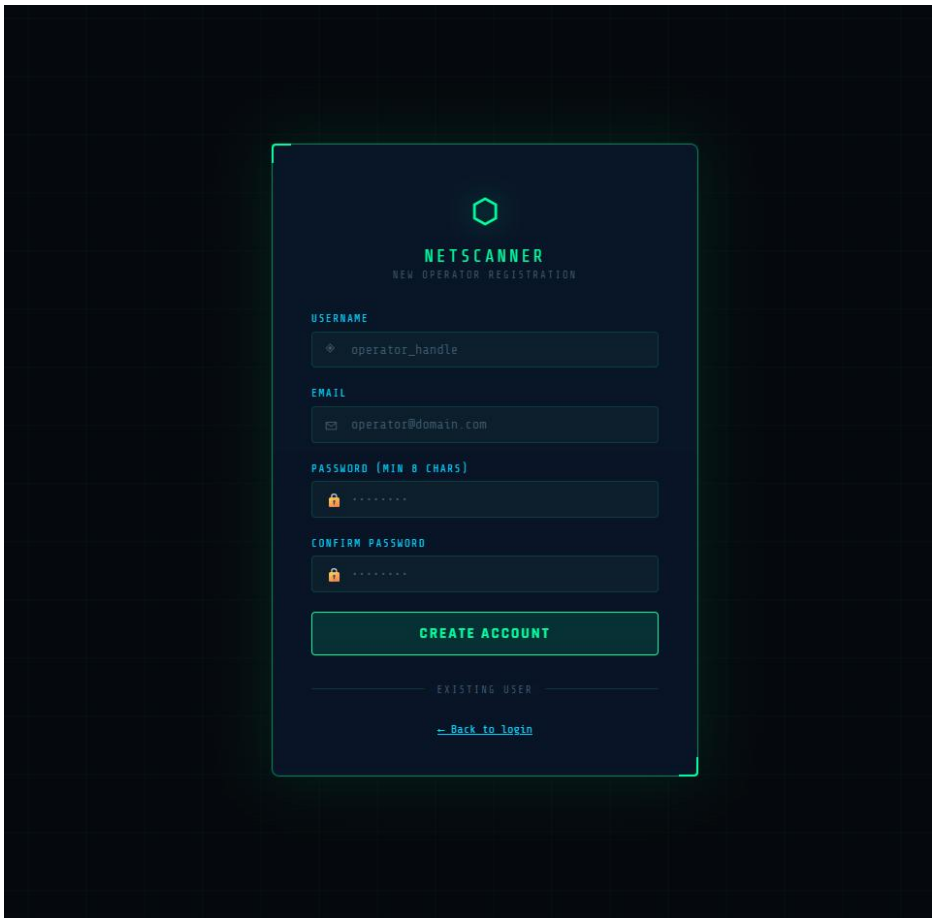

Figure 8-9 Sign Up Page

- **Dashboard Interface**

  The **Dashboard Interface** of the NetScanner platform provides a centralized overview of network scanning activities and system status. It serves as the main workspace where users can monitor scan progress, review results, and navigate between different scanning modules.

  The left-side navigation panel allows access to major features such as **Dashboard, Host Discovery, Port Scan, Network Radar, Log Analyzer, Results, Scheduler, and Profile**, ensuring easy navigation across the platform.

  The dashboard displays summary cards showing statistics like **Total Scans, Active Scans, and Scheduled Tasks**, along with graphical representations of scan activity trends and scan status distribution. A **Recent Scans table** presents key details such as target IP address, scan type, status, duration, and date, with a **View** option for detailed reports.

  Overall, the interface presents scanning information in a clear and structured format, helping users efficiently monitor and manage network scanning operations.

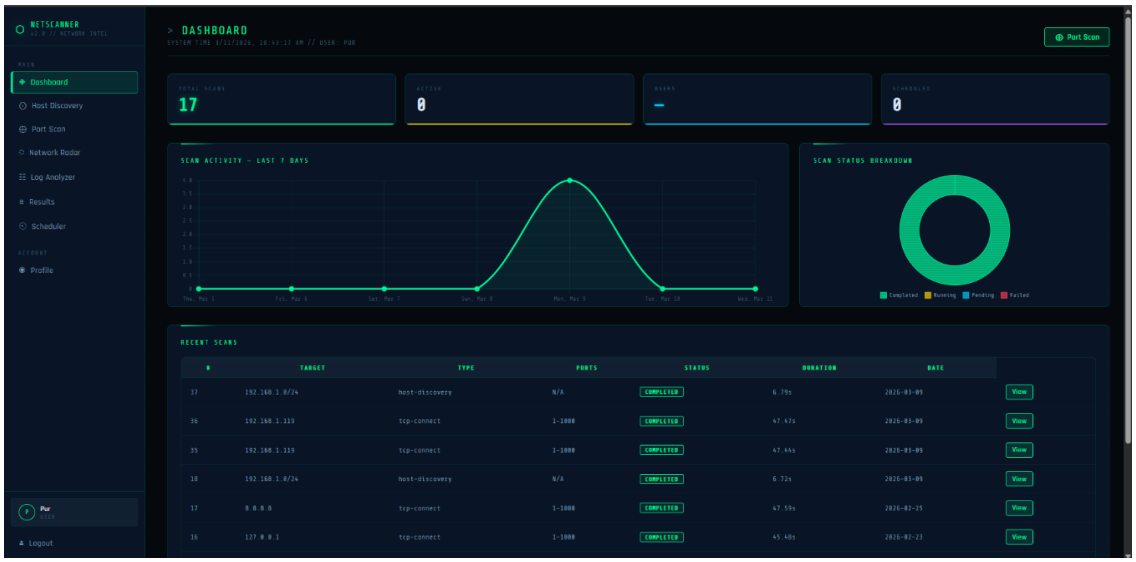

Figure 8-10 Dashboard

- **Scan Results Interface**

  The **Scan Results Interface** displays the outcomes of previously executed network scans in a structured table format. It provides filtering options such as **Scan Type**, **Status**, **Date Range**, and **Search Target**, allowing users to quickly locate specific scan records.

  Each entry shows important details including **Target Address**, **Scan Type**, **Port Range**, **Scan Status**, **Progress**, **Duration**, and **Date**. Action buttons such as **View**, **Topology**, and **Delete** allow users to analyze scan reports, visualize network structure, or manage stored results efficiently.

  This interface helps users organize, review, and manage scanning history in a clear and user-friendly manner.

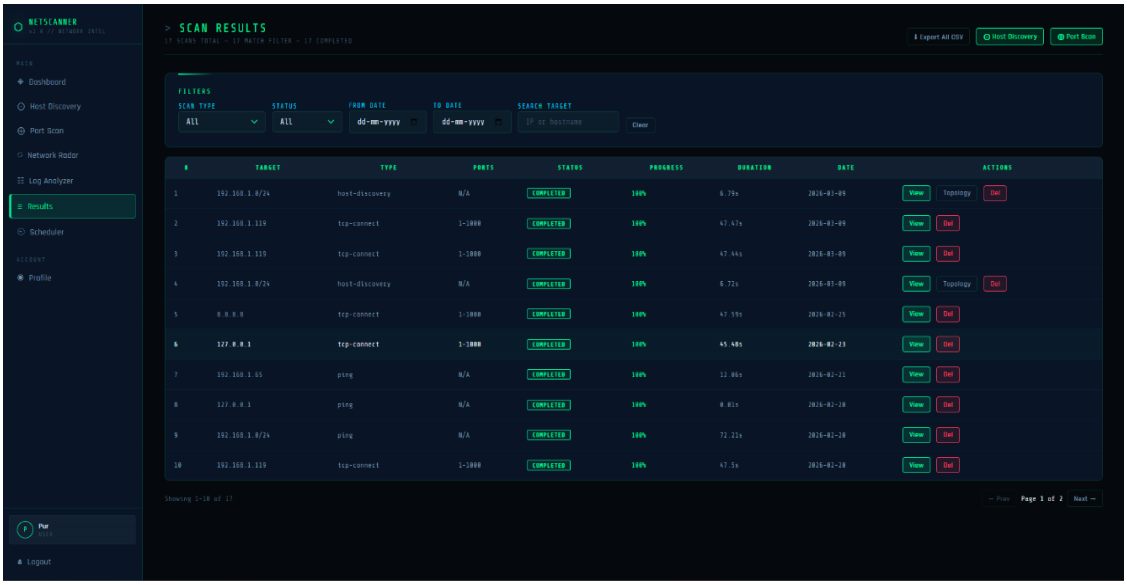

Figure 8-11 Result Page

- **Host Discovery Interface**

  The **Host Discovery Interface** enables users to identify active hosts within a specified network range. Users can enter either a **Start IP and End IP range** or a **CIDR notation** to perform discovery operations.

  After initiating the scan using the **Start Discovery** button, the system identifies reachable hosts within the selected network segment and displays the results in the output section. A **Scan Selected Hosts** option allows users to perform further analysis on discovered devices.

  This interface simplifies network exploration by providing an easy method to detect active devices within a network environment.

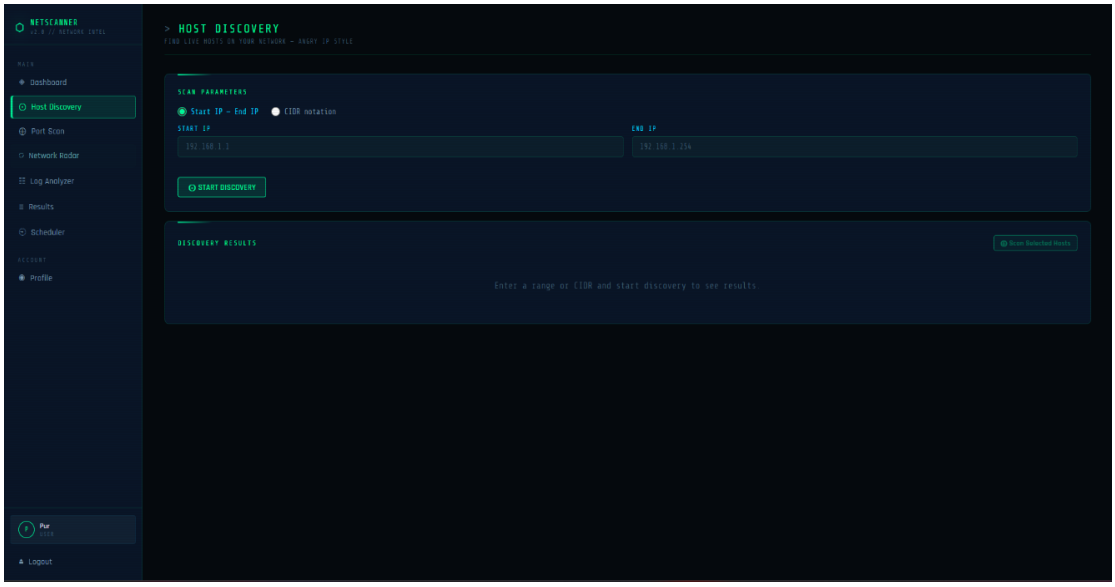

Figure 8-12 Host Discovery Page

- **Log Analyzer Interface**

  The **Log Analyzer Interface** allows users to upload and analyze network-related log files for further inspection and understanding. Users can select a log file using the **Choose File** option and initiate processing through the **Analyze Log** button.

  After processing, the system extracts useful information from the uploaded logs and presents meaningful insights that assist users in identifying network behavior and potential security-related events.

  This interface improves the usability of log data by converting complex log entries into understandable information for analysis.

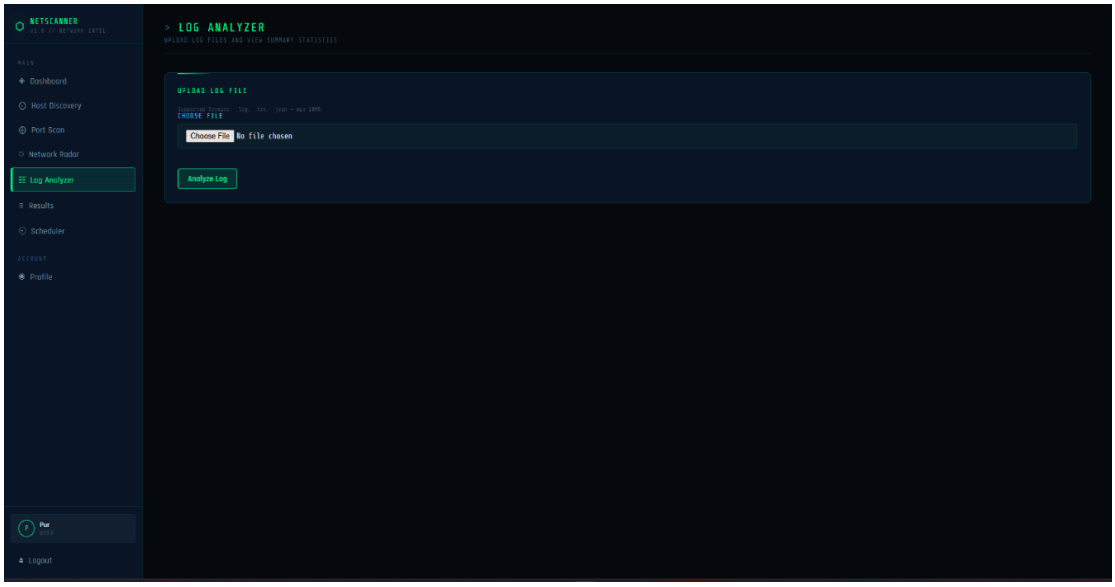

Figure 8-13 Log Analyzer Page

- **Port Scan Interface**

  The **Port Scan Interface** allows users to configure and execute port scanning operations on selected targets. Users can specify the **Target Type** (public or private), enter one or multiple **Target IP addresses**, define the **Port Range**, and select the **Scan Type** and **Scan Mode**.

  Additional configuration options such as **Timeout** and **Threads** enable users to control scan performance and execution behavior. A **Launch Scan** button starts the scanning process, while a **Terminal Output panel** displays real-time scan activity and results.

  This interface provides flexible and controlled scanning capabilities through an organized and easy-to-use configuration layout, supporting efficient network security analysis.

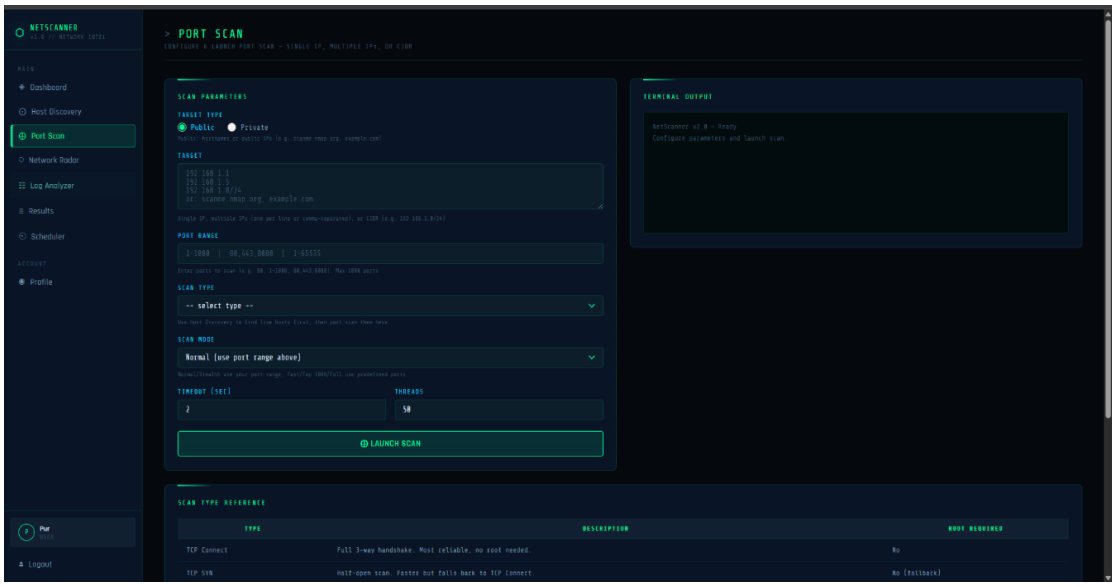

Figure 8-14 Port Scan Page

9. IMPLEMENTATION DETAILS

9.1. Algorithm of Implementation

The Web-Based Network Scanner follows a structured algorithm to perform network scanning, process results, and display them on the web interface.

Algorithm: Network Scanning and Analysis Process

Step 1: User Authentication

1. Start the system
2. User logs in using valid credentials
3. If authentication fails → display error and retry
4. If successful → redirect to dashboard

Step 2: Input Collection

5. User selects scan parameters:
   - Target IP / Domain / IP Range
   - Scan type (e.g., TCP Scan, SYN Scan)
   - Service detection option
6. Validate user input
7. If invalid → show error message
8. If valid → proceed

Step 3: Host Discovery

9. Send ICMP/ARP requests to identify live hosts
10. Store list of active hosts

Step 4: Port Scanning

11. For each active host:

- Scan specified ports (or default range)
- Classify ports as:
  - Open
  - Closed

- Filtered

Step 5: Service Detection

12. For each open port:

- Identify running service (HTTP, FTP, SSH, etc.)
- Detect protocol details

Step 6: Data Processing

13. Collect scan results
14. Convert raw data into structured format (JSON / DB format)
15. Store results in database

Step 7: Visualization

16. Display results on dashboard:

- Tables (hosts, ports, services)
- Graphs (open vs closed ports, etc.)

Step 8: Logging and Reporting

17. Save scan history
18. Allow user to view previous scans

Step 9: End Process

19. End

9.2. Flowchart of Implementation

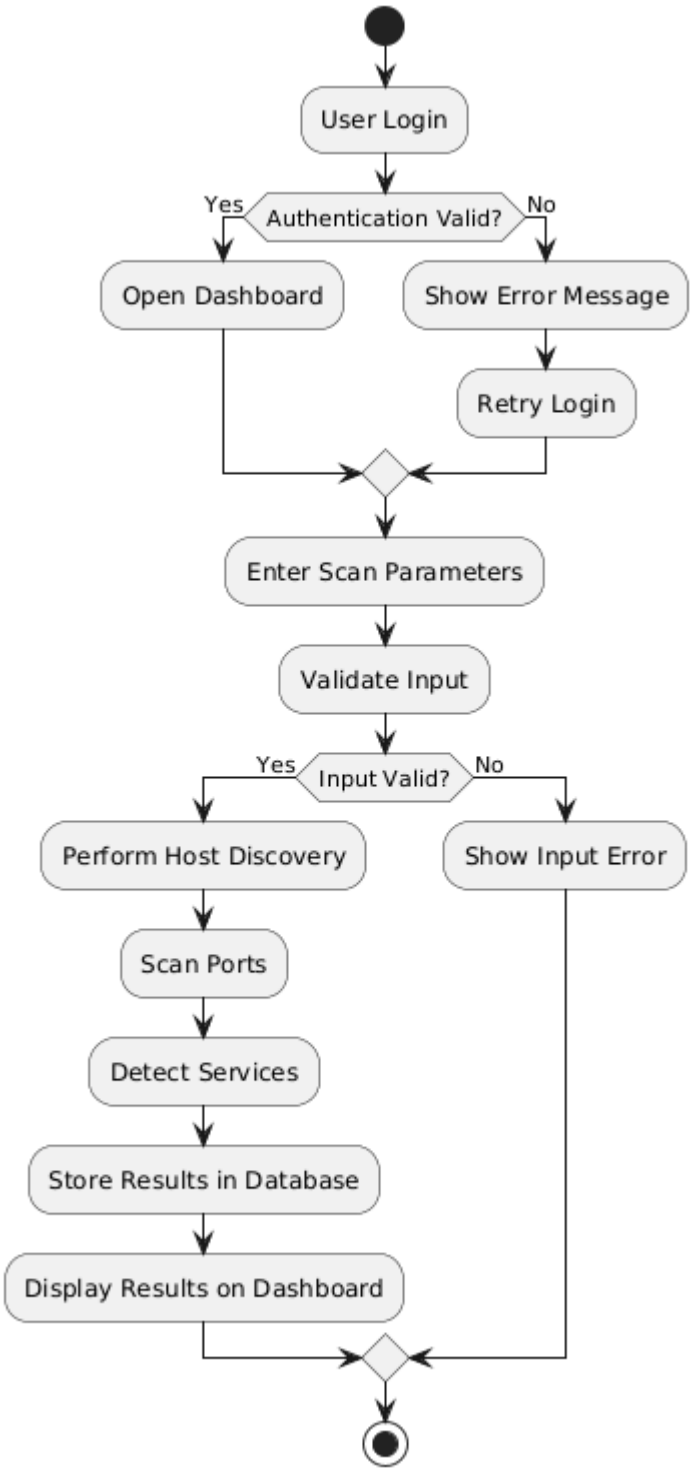

Figure 9-1 Flowchart

9.3. Actual Program Code

- Host Discovery Code

```
import ipaddress
import logging
import platform
import re
import socket
import subprocess
from concurrent.futures import ThreadPoolExecutor, as_completed


logger = logging.getLogger(__name__)


# Angry IP–style: short timeout for fast sweep (dead hosts don't block)
PING_TIMEOUT_MS = 300   # Windows: -w 300ms
PING_TIMEOUT_SEC = 1    # Linux/macOS: -W 1 (min practical)
PING_COUNT = 1
MAX_WORKERS = 200       # High concurrency for /24 (~254 hosts) in 1–2 waves
# Max hosts to prevent runaway scans (e.g. /16 = 65536 hosts)
MAX_HOSTS = 1024


def _generate_ips_from_range(start_ip: str, end_ip: str) -> list[str]:
    """Generate list of IPv4 addresses from start to end inclusive."""
    try:
        start = ipaddress.ip_address(start_ip)
```

```
        end = ipaddress.ip_address(end_ip)
        if start.version != 4 or end.version != 4:
            return []
        if start > end:
            start, end = end, start
        ips = []
        current = start
        while current <= end and len(ips) < MAX_HOSTS:
            ips.append(str(current))
            current += 1
        return ips
    except ValueError:
        return []


def _generate_ips_from_cidr(cidr: str) -> list[str]:
    """Generate list of IPv4 addresses from CIDR (e.g. 192.168.1.0/24)."""
    try:
        net = ipaddress.ip_network(cidr.strip(), strict=False)
        if net.version != 4:
            return []
        hosts = list(net.hosts())
        return [str(h) for h in hosts[:MAX_HOSTS]]
    except ValueError:
        return []
```

```
def _resolve_hostname(ip: str) -> str | None:
    """Reverse DNS lookup with short timeout. Returns hostname or None."""
    try:
        hostname, _, _ = socket.gethostbyaddr(ip)
        return hostname
    except (socket.herror, socket.gaierror, socket.timeout, OSError):
        return None


def _ping_host(ip: str) -> dict:
    """
    Ping a single host. Returns dict with ip, status, latency (ms), hostname.
    Hostname resolution is deferred to avoid blocking; caller runs it in batch.
    """
    result = {"ip": ip, "status": "dead", "latency": None, "hostname": None}


    try:
        if platform.system().lower() == "windows":
            cmd = ["ping", "-n", str(PING_COUNT), "-w", str(PING_TIMEOUT_MS), ip]
            proc_timeout = (PING_TIMEOUT_MS / 1000) + 0.5
        else:
            cmd = ["ping", "-c", str(PING_COUNT), "-W", str(PING_TIMEOUT_SEC), ip]
            proc_timeout = PING_TIMEOUT_SEC + 1


        proc = subprocess.run(
```

```
            cmd,
            capture_output=True,
            text=True,
            timeout=proc_timeout,
        )

        output = (proc.stdout or "") + (proc.stderr or "")

        latency_match = re.search(r"time[=<](\d+(?:\.\d+)?)\s*ms", output, re.I)
        if latency_match:
            result["latency"] = f"{float(latency_match.group(1)):.0f}ms"

        if proc.returncode == 0 or "ttl=" in output.lower() or "reply from" in output.lower():
            result["status"] = "alive"
        else:
            result["status"] = "dead"
            if not result["latency"]:
                result["latency"] = "Timeout"

    except subprocess.TimeoutExpired:
        result["status"] = "dead"
        result["latency"] = "Timeout"
    except Exception as e:
        logger.debug("Ping error for %s: %s", ip, e)
        result["status"] = "dead"
```

```
        result["latency"] = "Error"

    return result

def _resolve_hostnames_for_alive(results: list[dict]) -> None:
    """Resolve hostnames for alive hosts in parallel (mutates results in place)."""
    alive = [r for r in results if r["status"] == "alive"]
    if not alive:
        return

    old_timeout = socket.getdefaulttimeout()
    socket.setdefaulttimeout(0.5)
    try:
        def do_resolve(r: dict) -> None:
            hn = _resolve_hostname(r["ip"])
            if hn:
                r["hostname"] = hn

        with ThreadPoolExecutor(max_workers=min(50, len(alive))) as exe:
            list(exe.map(do_resolve, alive))
    finally:
        socket.setdefaulttimeout(old_timeout)

def run_discovery(
    start_ip: str | None = None,
```

```
    end_ip: str | None = None,
    cidr: str | None = None,
) -> list[dict]:
    """
    Run host discovery scan.
    Provide either (start_ip + end_ip) OR cidr.
    Returns list of dicts: { ip, status, latency, hostname }
    """
    ips: list[str] = []

    if cidr:
        ips = _generate_ips_from_cidr(cidr)
    elif start_ip and end_ip:
        ips = _generate_ips_from_range(start_ip, end_ip)
    else:
        return []

    if not ips:
        return []

    results: list[dict] = []
    total = len(ips)

    workers = min(MAX_WORKERS, total)
    with ThreadPoolExecutor(max_workers=workers) as executor:
```

```
        futures = {executor.submit(_ping_host, ip): ip for ip in ips}
        for future in as_completed(futures):
            try:
                r = future.result()
                results.append(r)
            except Exception as e:
                ip = futures.get(future, "?")
                logger.debug("Discovery error for %s: %s", ip, e)
                results.append({
                    "ip": str(ip),
                    "status": "dead",
                    "latency": "Error",
                    "hostname": None,
                })

    # Deferred hostname resolution (parallel, doesn't block ping phase)
    _resolve_hostnames_for_alive(results)

    results.sort(key=lambda x: ipaddress.ip_address(x["ip"]))
    return results
```

- Port Scanning Code

```
import socket
import logging
from concurrent.futures import ThreadPoolExecutor, as_completed
from typing import Callable, Optional
```

```
from backend.services.scanner_engine.timing_engine import (
    delay_between_probes,
    should_delay_between_probes,
)

logger = logging.getLogger(__name__)

def _tcp_connect(
    host: str,
    port: int,
    timeout: float,
    grab_banner: bool,
) -> dict | None:
    """
    Single TCP connect probe. Returns port dict on success, None on failure.
    If grab_banner=True, read first bytes for later service detection.
    """
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.settimeout(timeout)
            s.connect((host, port))
            banner = ""
            if grab_banner:
                try:
```

```
                s.settimeout(1.0)
                raw = s.recv(256)
                banner = raw.decode("utf-8", errors="replace").strip()[:120]
            except Exception:
                pass
        return {
            "port": port,
            "protocol": "tcp",
            "state": "open",
            "banner": banner,
        }
    except (socket.timeout, ConnectionRefusedError, OSError):
        return None


def run_connect_scan(
    hosts: list[str],
    ports: list[int],
    timeout: float = 2.0,
    timing_profile: str = "T3",
    max_workers: int = 50,
    grab_banner: bool = True,
    progress_cb: Optional[Callable[[int], None]] = None,
) -> dict[str, list[dict]]:
    """
    Run TCP connect scan on each host for the given ports.
```

```
Uses ThreadPoolExecutor; optional per-probe delay based on timing_profile.


Returns:
    { "host_ip": [ {"port", "protocol", "state", "banner"}, ... ], ... }
"""
do_delay = should_delay_between_probes(timing_profile)
total = len(hosts) * len(ports)
completed = [0]  # mutable for closure


result_by_host: dict[str, list[dict]] = {h: [] for h in hosts}


def probe_one(host: str, port: int) -> Optional[dict]:
    r = _tcp_connect(host, port, timeout, grab_banner=grab_banner)
    if do_delay:
        delay_between_probes(timing_profile)
    return r


for host in hosts:
    with ThreadPoolExecutor(max_workers=max_workers) as exe:
        futures = {exe.submit(probe_one, host, p): (host, p) for p in ports}
        for future in as_completed(futures):
            host, port = futures[future]
            completed[0] += 1
            if progress_cb and total > 0:
                pct = int((completed[0] / total) * 100)
```

```
            progress_cb(pct)
        try:
            r = future.result()
            if r:
                result_by_host[host].append(r)
        except Exception as e:
            logger.debug("Probe error %s:%s: %s", host, port, e)

    for host in result_by_host:
        result_by_host[host].sort(key=lambda x: x["port"])
    return result_by_host
```

10. TESTING

10.1. Testing Approach

The system was tested using a combination of **manual testing and functional validation techniques** to ensure that all modules perform as expected. Each component of the system, including host discovery, port scanning, service detection, backend APIs, and database operations, was tested independently and then integrated for system-level testing.

Testing was conducted in a controlled environment using local and test networks to ensure ethical and safe execution of network scanning operations.

10.2. Testing Model Used

The project follows a combination of Unit Testing and Integration Testing within the Agile Incremental Model.

- **Unit Testing:**
  Individual modules such as host discovery, port scanning, and service detection were tested independently to verify their correctness.
- **Integration Testing:**
  After individual modules were validated, they were integrated and tested together to ensure proper communication between components such as frontend, backend, and database.
- **System Testing:**
  The complete system was tested as a whole to validate end-to-end functionality, including user input, scanning process, and result visualization.

This approach ensured early detection of errors and continuous improvement during development.

10.3. Types of Testing Performed

The following types of testing were carried out:

1. **Functional Testing**
   - Verified all system functionalities such as scanning, login, and result display.
   - Ensured that each feature works according to requirements.
2. **Performance Testing**
   - Tested system response time during scanning operations.
   - Evaluated performance for multiple hosts and ports.
3. **Security Testing**
   - Verified authentication and role-based access control.
   - Ensured protection against invalid inputs and unauthorized access.
4. **Usability Testing**
   - Evaluated ease of use of the cyberpunk-themed web interface.
   - Ensured clarity and responsiveness across devices.

10.4. Test Cases and Results

Table 10-1 Test Cases

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC01	User Login	Valid credentials	Redirect to dashboard	Successful login	Pass
TC02	Invalid Login	Wrong password	Error message	Error displayed	Pass
TC03	Host Discovery	Valid IP range	List of active hosts	Hosts detected	Pass

TC04	Port Scanning	Target IP + ports	Open/closed ports	Ports identified correctly	Pass
TC05	Service Detection	Open ports	Service details	Services detected	Pass
TC06	Invalid Input	Incorrect IP	Error message	Validation works	Pass
TC07	Save Results	Scan results	Data stored in DB	Stored successfully	Pass
TC08	View Results	Previous scan	Display results	Results displayed	Pass
TC09	Performance	Large scan	Acceptable delay	Slight delay but acceptable	Pass
TC10	Authorization	Unauthorized user	Access denied	Access restricted	Pass

10.5. Test Results Summary

The testing results indicate that the system performs all intended functionalities accurately and reliably. All core modules, including host discovery, port scanning, service detection, and data storage, function as expected.

The system demonstrated stable performance under normal operating conditions. Minor delays were observed during large-scale scans, which are expected due to the nature of network operations. Security mechanisms such as authentication and input validation were successfully implemented and tested.

Overall, the system meets the functional and non-functional requirements specified in the SRS.

10.6. Conclusion of Testing

The testing phase confirms that the Web-Based Network Scanner is reliable, efficient, and user-friendly. The use of systematic testing methods ensured early detection of issues and continuous improvement of the system.

The application is ready for deployment in controlled environments and is suitable for academic use and network analysis tasks.

11.USER MANUAL

11.1. About Web-Based Network Scanner

The **Web-Based Network Scanning and Analysis System** is designed to provide an efficient, accessible, and user-friendly platform for discovering devices and services within a network environment. The system enables users to perform various network scanning operations through a web-based interface, eliminating the need for complex command-line tools and making network analysis more approachable.

The primary objective of the system is to assist users in **network discovery, monitoring, and basic security assessment**. It allows users to identify active hosts, detect open ports, and gather essential network-related information. By providing insights into the structure and behavior of a network, the system helps improve overall network visibility and management.

The application enables users to configure scanning parameters such as target IP address, hostname, or network range, along with scan-specific options. Once configured, users can initiate scans directly from the web interface. The backend system processes these requests, executes the scanning operations, and collects the results in a structured format.

The scan results are presented through an **interactive dashboard**, which includes tabular data, summaries, and visual representations. This structured presentation allows users to easily interpret the results, understand network topology, and identify potential vulnerabilities or misconfigurations.

The scope of the system includes key functionalities such as **host discovery, port scanning, service identification, result visualization, and log analysis**. These features work together to provide a comprehensive view of network activity and system exposure.

The system is intended for use in **controlled and authorized environments**, ensuring ethical application of network scanning techniques. It is particularly suitable for **academic learning, administrative monitoring, and basic security**

assessments, making it a valuable tool for students, network administrators, and cybersecurity professionals.

11.2. System Requirements

The following hardware and software requirements define the minimum specifications needed to run the Web-Based Network Scanner application effectively.

1. Hardware Requirements

The system does not require high-end hardware and can operate on a standard computer system.

- **Processor:** Intel Core i3 or equivalent
- **RAM:** Minimum 4 GB (8 GB recommended for better performance)
- **Storage:** At least 1 GB of free disk space
- **Network Interface:** Ethernet or Wi-Fi connection for network scanning operations

2. Software Requirements

The application is developed using modern web technologies and requires the following software components:

- **Programming Language:** Python 3.10 or higher
- **Web Framework:** Flask
- **Database:** SQLite
- **Network Scanning Tools:** Nmap
- **Additional Libraries:** Flask-SocketIO, APScheduler, and other Python dependencies listed in requirements.txt

3. Operating System Requirements

The system can run on multiple operating systems that support Python and the required dependencies:

- Windows 10 / Windows 11
- Linux (Ubuntu, Kali Linux, Debian, etc.)
- macOS

4. Browser Requirements

Since the system is web-based, it can be accessed through any modern web browser that supports JavaScript and modern web standards.

Recommended browsers include:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

5. Network Requirements

- Local network access for scanning hosts and services
- Internet connection (optional) for updates or external integrations

11.3. User Types

The system supports different categories of users based on their level of access and functionality within the application.

- Registered User

  A registered user is an authenticated user who has created an account in the system. Registered users can access all available features of the application, including initiating network scans, viewing scan results, analyzing logs, and managing their scan history.

  Currently, the system operates primarily with registered users, and all scanning activities require authentication.

11.4. Registration (New Users)

New users can create an account to access the network scanning platform.

The registration process involves the following steps:

1. The user opens the registration page of the application.

2. The user enters required information such as username, email address, and password.
3. The system validates the provided information.
4. If the information is valid, the account is created and stored in the database.
5. The user can then log in using the registered credentials.

This process ensures that only authorized users can access the system features.

11.5. Login

The login functionality allows registered users to securely access the system.

The login process works as follows:

1. The user navigates to the login page.
2. The user enters their registered username and password.
3. The system verifies the credentials against the stored data in the database.
4. If authentication is successful, the user is redirected to the dashboard.
5. If authentication fails, an error message is displayed to the user.

This authentication mechanism ensures that only authorized users can access the system.

11.6. User Portal Guide

The user portal provides an interface through which registered users can perform network scanning and analysis tasks.

- Initiating network scans
  Users can initiate scans by entering a target IP address, hostname, or network range and selecting the desired scanning options.
- Viewing scan results
  After the scan is completed, the results are displayed in the results dashboard. The system shows information such as open ports, active hosts, and scan status.
- Managing scan history

The system maintains a history of previous scans. Users can review past scan results, filter scans, and export results when needed.

11.7. Network Scanning and Analysis Features

Details the core functionality of the system including:

- **Host discovery**

  The host discovery feature enables the system to identify active devices within a specified network or IP range. This process is essential as it determines which hosts are reachable and responsive before initiating further scanning operations.

  The system performs host discovery using techniques such as ICMP (ping requests), ARP requests, or other probing methods to detect live hosts. Only those devices that respond to these probes are considered active and are included in subsequent scanning processes.

  This feature helps reduce unnecessary scanning of inactive or unreachable systems, thereby improving efficiency and reducing scan time.

- **Port scanning**

  The port scanning feature allows the system to examine target devices and identify the status of various ports. Ports are categorized as **open, closed, or filtered**, providing insight into the accessibility of services on a given host.

  The system supports different scanning techniques such as TCP Connect Scan or SYN Scan, depending on implementation. By analyzing port states, users can understand which services are exposed to the network and may require further investigation.

  This functionality is critical for detecting potential entry points into a system and plays a key role in network security assessment.

- **Service detection**

The service detection feature enhances the port scanning process by identifying the services running on open ports. Once open ports are detected, the system attempts to determine the corresponding service (e.g., HTTP, FTP, SSH) using techniques such as port-to-service mapping or banner grabbing.

This feature provides deeper insight into the network environment by revealing the type of services available and their potential versions or configurations. Such information is useful for identifying vulnerabilities and understanding the behavior of networked systems.

- **Scan result visualization**

  The system presents scan results through an interactive and user-friendly dashboard. The visualization component transforms raw scan data into structured and meaningful formats, making it easier for users to interpret the results.

  The dashboard may include:

  - Tabular representations of hosts, ports, and services
  - Summary statistics of scan results
  - Graphical charts and visual indicators
  - Network topology or relationship views (if implemented)

11.8. Troubleshooting

- **Login Errors**

  Users may encounter login issues due to incorrect credentials or authentication problems. In such cases, it is important to ensure that the correct username and password are entered. Users should verify that there are no typing errors, such as incorrect capitalization or extra spaces.

  If the issue persists, it is recommended to check whether the user account exists in the system database and has been properly registered. In case of forgotten credentials, users should follow the password recovery mechanism (if implemented) or contact the system administrator for assistance.

- **Scan Failures**

  Scan failures may occur when the system is unable to initiate or complete a network scan. This can happen due to invalid input or lack of connectivity with the target system.

  Users should ensure that the target IP address, domain name, or network range entered is valid and correctly formatted. Additionally, the scanning system must have proper network access to the target. If the target system is offline or unreachable, the scan may fail or return incomplete results.

- **Connectivity Problems**

  Network connectivity issues can significantly affect the performance and accuracy of scanning operations. Users should verify that the system is properly connected to a network, either through Ethernet or Wi-Fi.

  It is also important to check firewall and security settings, as these may block scanning requests or restrict certain types of network traffic. Disabling restrictive firewall rules (in a controlled and safe environment) or allowing necessary permissions can help resolve connectivity-related issues.

- **Slow Scan Performance**

  Scanning performance may vary depending on factors such as network size, number of ports scanned, and system resources. When scanning large IP ranges or performing detailed service detection, the process may take additional time.

  To improve performance, users are advised to:

  - Limit the scan range to specific IP addresses or smaller subnets
  - Reduce the number of ports being scanned
  - Avoid running multiple scans simultaneously
  - Ensure sufficient system resources (CPU and RAM) are available

  These measures help optimize scanning speed while maintaining accuracy.

- **General Recommendations**

- Always perform scans in authorized and controlled environments
- Ensure the system is updated with required dependencies
- Regularly check logs for debugging and error analysis
- Contact the administrator for persistent or critical issues

12.CONCLUSION & FUTURE WORK

12.1. Conclusion

The project successfully presents the design and implementation of a **Web-Based Network Scanning and Analysis System** aimed at simplifying network discovery and monitoring processes. The system provides an efficient and user-friendly platform that enables users to identify active hosts, detect open ports, and analyze services within a network environment through an interactive web interface.

The application automates complex network scanning operations and eliminates the need for command-line interaction, making it accessible to both technical and non-technical users. By integrating functionalities such as host discovery, port scanning, service detection, and result visualization, the system offers a comprehensive solution for understanding network structure and behavior.

The use of a **modular architecture** ensures that the system is scalable, maintainable, and capable of supporting future enhancements. The integration of backend scanning logic with a dynamic frontend dashboard improves usability while maintaining accuracy and reliability in scanning results.

This project demonstrates the practical application of key networking and cybersecurity concepts, including network reconnaissance, port analysis, service identification, and data visualization. It also provides hands-on experience with modern development technologies such as Python, web frameworks, and database systems.

Furthermore, the system adheres to ethical and controlled scanning practices, ensuring that it is used responsibly within authorized environments. All functional and non-functional requirements defined in the SRS have been successfully implemented and validated through systematic testing.

In conclusion, the proposed system serves as an effective tool for **network analysis, academic learning, and basic security assessment**, particularly in small to medium-scale environments. It bridges the gap between complex

traditional tools and user-friendly modern applications, making network scanning more accessible and efficient.

12.2. Future Work

Although the proposed Web-Based Network Scanning and Analysis System successfully achieves its intended objectives, several enhancements can be implemented to further improve its functionality, scalability, and effectiveness in real-world scenarios.

1. Integration of Advanced Analysis Techniques
   The system can be enhanced by integrating advanced analysis techniques such as **machine learning-based network behavior analysis**. This would allow the system to identify unusual patterns in scan results, detect anomalies, and provide intelligent insights beyond basic port and service detection.

2. Real-Time Network Monitoring
   Currently, the system performs scanning on demand. In future versions, it can be extended to support **real-time network monitoring**, where continuous scanning and tracking of network changes are performed. This would enable users to detect newly connected devices, changes in port states, or service updates instantly.

3. Cloud Deployment and Scalability
   The system can be deployed on **cloud platforms** such as AWS, Azure, or Google Cloud to improve scalability and accessibility. Cloud deployment would allow the system to handle large-scale networks, support multiple concurrent users, and perform distributed scanning across different network environments.

4. Integration with External APIs and Threat Intelligence
   Future enhancements may include integration with external services such as **Shodan, AbuseIPDB, or VirusTotal APIs**. This would enable the system to provide additional intelligence about scanned hosts, such as reputation, known vulnerabilities, and threat indicators.

5. Advanced Visualization and Reporting

The visualization component can be further improved by adding **interactive graphs, network topology maps, and detailed reporting features**. Users could generate downloadable reports (PDF/CSV) with summarized insights, making the system more useful for documentation and analysis purposes.

6. Automated Vulnerability Detection

   The system can be extended to include **basic vulnerability scanning capabilities**, where detected services and ports are analyzed against known vulnerabilities. This would transform the system from a simple scanner into a more comprehensive security assessment tool.

7. Enhanced User Interface and Experience

   Further improvements in the **cyberpunk-themed user interface** can include advanced UI components, real-time updates, and customizable dashboards. Enhancing responsiveness and interactivity will improve overall user experience.

8. Mobile and Remote Access Support

   Future versions of the system can support **mobile compatibility and remote monitoring**, allowing users to initiate scans and view results from smartphones or remote devices. Notifications and alerts can also be implemented for completed scans or important findings.

9. Summary

   The future enhancements aim to transform the current system into a more **intelligent, scalable, and feature-rich network analysis platform**. By incorporating advanced technologies and expanding its capabilities, the system can evolve into a powerful tool suitable for both academic and professional cybersecurity applications.

13.ANNEXURE

13.1. Glossary of Terms and Abbreviations

Table 13-1 Glossary of Terms and Abbreviation

Term / Abbreviation	Description
IDS	Intrusion Detection System – a system that monitors network or system activities for malicious behavior or policy violations.
IPS	Intrusion Prevention System – actively blocks detected threats.
SIEM	Security Information and Event Management – collects and analyzes security logs from multiple sources.
OSINT	Open Source Intelligence – publicly available data used for intelligence gathering.
ML	Machine Learning – algorithms that learn patterns from data to make predictions or decisions.
Anomaly Detection	Technique used to identify unusual patterns that may indicate cyber attacks.
Packet	A unit of data transmitted over a network.
Signature based Detection	Detection based on known attack patterns or signatures.
Behavior-based Detection	Detection based on abnormal behavior rather than known signatures.
False Positive	A benign activity incorrectly identified as malicious.
False Negative	A malicious activity incorrectly identified as benign.
API	Application Programming Interface – allows communication between software components.
GUI	Graphical User Interface – visual interface for user interaction.
DFD	Data Flow Diagram – shows data movement within a system.
UML	Unified Modeling Language – used for system design diagrams.

TCP/IP	Transmission Control Protocol / Internet Protocol – fundamental networking protocols.
PCAP	Packet Capture file format used for network traffic analysis.

13.2. References

The following references were used during the design and development of the system:

1. Stallings, W., Network Security Essentials: Applications and Standards, Pearson Education.
2. Scapy Documentation – Packet manipulation and analysis framework.
3. Python Official Documentation – https://docs.python.org
4. OWASP Foundation – OWASP Top 10 Web Application Security Risks.
5. NIST Special Publication 800-94 – Guide to Intrusion Detection and Prevention Systems.
6. IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
7. Wireshark User Guide – Network protocol analysis documentation.
8. Research papers on anomaly-based intrusion detection using machine learning.

13.3. About Tools and Technology

- Programming Language - Python
    - Chosen for its simplicity, readability, and strong support for networking and security-related development.
    - Provides extensive libraries for network scanning, packet analysis, and backend web development.
    - Enables rapid prototyping and easy integration with web frameworks and system-level tools.
    - Networking and Scanning Libraries
- Scapy
    - Used for low-level packet manipulation and network probing.

    - Supports host discovery, packet crafting, and network analysis operations.
- Socket Programming
    - Enables implementation of basic port scanning and network communication.
    - Helps in detecting open ports and active services on target systems.
- PyShark / PCAP Processing
    - Used for analyzing captured network traffic.
    - Assists in examining packets for protocol and communication details when required.
    - Machine Learning (Optional / Advanced Module)
- Scikit-learn
    - Can be used for implementing basic anomaly detection on network scan results.
    - Helps identify unusual patterns or suspicious network behavior.
- Pandas and NumPy
    - Used for data processing, analysis, and feature extraction.
    - Helps organize scan results and prepare data for analysis or reporting.
- Database - SQLite / PostgreSQL
    - Used to store scan results, logs, user activities, and historical data.
    - SQLite is suitable for lightweight deployments, while PostgreSQL supports scalability
    - Ensures efficient data retrieval and persistence.
- User Interface
    - Web-Based Dashboard (Flask / Django)
    - Provides a browser-accessible interface for initiating scans and viewing results.
    - Displays scanned hosts, open ports, detected services, and logs.
    - Includes administrative controls and reporting features.
- Operating System- Linux (Preferred)
    - Provides better support for low-level network operations and packet handling.

    - Commonly used in cybersecurity and network analysis environments.
- Windows
    - Supported for development, testing, and demonstration purposes.
- Development Tool - VS Code / Cursor IDE
    - Used for writing, debugging, and managing the project codebase.
- Git and GitHub
    - Used for version control and collaborative development.
    - Helps track changes and manage project versions.

13.4. About The Organization

- **Overview of the Organization**

  LetsDefend Infosec is a cybersecurity company that provides advanced security solutions aimed at protecting digital infrastructure, applications, and data from emerging cyber threats. With the rapid expansion of digital technologies and online services, organizations face increasing challenges in maintaining the security of their information systems. LetsDefend Infosec focuses on delivering specialized cybersecurity services that help organizations identify vulnerabilities, strengthen defenses, and maintain compliance with security standards.

  The company is founded and managed by professionals with significant experience in the field of information security. By combining technical expertise with modern security technologies, LetsDefend Infosec assists businesses in safeguarding their digital assets and maintaining the confidentiality, integrity, and availability of critical information systems.

- **Mission and Vision**

  The mission of LetsDefend Infosec is to provide reliable and effective cybersecurity services that help organizations proactively defend against cyber threats. The company aims to strengthen the overall security posture of its clients by identifying vulnerabilities, monitoring potential threats, and implementing robust security measures.

The vision of the organization is to become a trusted cybersecurity partner for organizations by offering innovative and scalable security solutions that address evolving cyber risks.

- **Services Offered**

  LetsDefend Infosec offers a wide range of cybersecurity services designed to protect organizations from various digital threats. Some of the major services provided by the organization include:

  1. **Virtual CISO as a Service**
     - Governance oversight
     - Threat assessment
     - Security consulting
     - Security integration
     - Compliance analysis and liaison
  2. **Vulnerability Assessment and Penetration Testing (VAPT)**

     The organization provides comprehensive testing services to identify security weaknesses in systems and applications. These include:
     - Web application VAPT
     - Mobile application VAPT
     - Internal and external network VAPT
     - Server VAPT
     - Endpoint VAPT
     - Cloud VAPT
     - IoT device VAPT
  3. **Audit and Compliance Services**

     LetsDefend Infosec also provides auditing and compliance support to ensure that organizations follow industry security standards. These services include:
     - Information system audit
     - Cybersecurity audit
     - Data security audit
     - Ransomware readiness review
     - Supply chain risk management review

- Third-party security posture assessment
- Risk-based internal audit

4. **Monitoring as a Service**

   The company offers continuous monitoring solutions to detect and respond to potential threats. These services include:

   - Attack surface management
   - Security Operations Center (SOC) as a Service
   - Dark web monitoring
   - Managed patch management
   - Anti-phishing and anti-rogue services
   - Privileged Access Management (PAM)

5. **Cybersecurity Awareness and Training**

   LetsDefend Infosec also provides training programs to improve cybersecurity awareness and defensive capabilities within organizations. These include:

   - Cybersecurity training for board and senior management
   - IT department cybersecurity training
   - Employee cybersecurity awareness training
   - IT security best practices training
   - Endpoint and server hardening training
   - Email phishing simulations
   - Ransomware attack simulations

- **Key Strengths of the Organization**

  LetsDefend Infosec is known for its strong focus on proactive cybersecurity strategies and customized security solutions. The organization emphasizes identifying potential risks before they lead to major security incidents.

  Some of the key strengths of the organization include:

  - Experienced cybersecurity professionals
  - Customized security solutions for different industries
  - Proactive threat detection and risk management
  - Scalable security services for businesses of all sizes
  - Cost-effective cybersecurity leadership and consulting

- **Importance of the Organization**

With the increasing number of cyber attacks and digital threats, organizations must adopt strong cybersecurity practices to protect their systems and sensitive data. Companies like LetsDefend Infosec play a crucial role in helping businesses strengthen their security posture by providing professional security assessments, monitoring services, and expert guidance. By offering advanced cybersecurity services, the organization contributes to building a safer and more resilient digital environment.

13.5. About College

- **Overview of the College**

  U. V. Patel College of Engineering (UVPCE) is a premier engineering institute located within the campus of Ganpat University, in Kherva, Mehsana district, Gujarat, India. The college was established with the objective of providing quality technical education and developing skilled professionals capable of meeting the demands of modern industries and technological advancements.

  UVPCE offers undergraduate engineering programs in several disciplines including Computer Engineering, Information Technology, Computer Science and Engineering, Artificial Intelligence, and other emerging technological fields. The institute focuses on providing a strong academic foundation along with practical knowledge and industry-oriented training.

- **Vision and Mission**

  The vision of UVPCE is to become a leading institution in engineering education by fostering innovation, research, and technical excellence. The college aims to produce competent engineers who can contribute effectively to technological development and societal progress.

  The mission of the institute is to provide high-quality education, encourage research and innovation, and develop ethical and responsible professionals who can address real-world engineering challenges.

- **Academic Programs**

  UVPCE offers a variety of undergraduate engineering programs designed to equip students with both theoretical knowledge and practical skills. These

programs emphasize modern technologies, project-based learning, and industry exposure.

The curriculum is regularly updated to incorporate advancements in fields such as computer science, information technology, artificial intelligence, cybersecurity, and data science.

- **Infrastructure and Facilities**

  The college provides modern infrastructure and facilities to support academic learning and research activities. The campus includes well-equipped laboratories, computer centers, classrooms, seminar halls, and library facilities. The laboratories are designed to provide hands-on experience in various engineering subjects and technologies.

  The college also provides internet-enabled computer labs, research facilities, and collaborative learning environments that help students gain practical knowledge and technical expertise.

- **Industry Interaction and Training**

  UVPCE emphasizes industry interaction and practical exposure for students. The college encourages students to participate in internships, industrial training, workshops, seminars, and technical events. These activities help students understand real-world applications of engineering concepts and prepare them for professional careers.

  The institute also promotes innovation and project-based learning through final-year projects, research activities, and collaboration with industry experts.

- **Contribution to Technical Education**

  Over the years, UVPCE has played an important role in providing quality technical education and producing skilled engineering graduates. Through its strong focus on academic excellence, research, and industry collaboration, the institute continues to contribute to the development of competent engineers and technological innovation.